

目录

第一章 Numpy.....	3
1. 广播.....	3
第二章 感知机.....	3
1. 什么是感知机？.....	3
2. 导入权重与偏置.....	3
3. 线性和非线性.....	4
4. 阶跃函数.....	4
第三章 神经网络.....	4
1. 什么是神经网络.....	5
2. 激活函数.....	5
3. 非线性函数.....	6
4. ReLU (Rectified Linear Unit) 函数.....	6
5. 神经网络的内积.....	6
6. softmax 函数：记住公式，实现归一，使输出结果加合为1.....	8
7. 批处理.....	9
第四章 神经网络的学习.....	9
1. 均方误差.....	10
2. one-hot 编码.....	10
3. 交叉熵误差.....	10
4. mini-batch.....	11
5. 为什么要设置损失函数？.....	11
6. 数值微分.....	11
7. 梯度.....	12
8. 学习算法的实现.....	13
9. 基于测试数据的评价.....	13
第五章 误差反向传播算法.....	13
1. 计算图法.....	13
2. 乘法层的实现.....	14
3. Relu 实现.....	15
4. Sigmoid 实现.....	15
5. Affine/Softmax 实现.....	15
6. softmax-with-loss.....	16
7. 误差反向传播算法的实现.....	17
第六章 神经网络的学习技巧.....	17
1. 参数更新.....	18
2. Momentum.....	18
3. AdaGrad.....	18
4. Adam.....	18

5. 权重的初始值.....	19
----------------	----

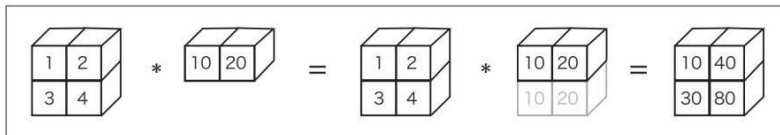
6. Relu 初始值	20
7. Batch Normalization.....	21
8. 正则化.....	22
9. 超参数验证	22
第七章 卷积神经网络	23
1. 全连接层	23
2. CNN 整体结构	23
3. 卷积层.....	24
4. 池化处理	26
第八章 深度学习.....	28
1. 数据扩充	28
2. 为什么加深层数.....	28
深度学习进阶：自然语言处理.....	28
第二章 自然语言和单词的分布表示	29
1. 共现矩阵、余弦相似度、点互信息、降维.....	29
第三章 word2vec.....	30
1 什么是word2vec	30
2 CBOW 怎么设计模型.....	30
3 概率表示的时候CBOW 是什么样的.....	31
4 skip-gram 模型，和CBOW 翻转	31
5 skip-gram 和CBOW 对比.....	32
第四章 word2vec 高速化	33
1. word2vec 计算优化，对输入更新，不需要对全部更新	33
2. 如果负样本太多，就用采样的方法，看一下负采样的步骤.....	34
3. 单词向量的评价方法.....	35
第五章 RNN.....	35
1. RNN 的结构，数据流通环路.....	36
2. RNNLM 的结构.....	37
3. 困惑度.....	38
4. 梯度消失、梯度爆炸.....	38
5. 为什么会梯度消失、梯度爆炸.....	38
6. 如何解决梯度爆炸：梯度裁剪.....	38
7. 如何解决梯度消失：LSTM 整个LSTM 都非常重要：加入输入门、输出门、遗忘门的整体结构。每个门对应的公式表达要清楚	38
8. 权重共享	41
第七章 基于RNN 生成文本.....	42
1. 输入一个词，给出概率最高的下一个词，再输入再输出.....	42
2. 知道Encoder 和Decoder，编码器结构，输出h 是一个固定长度向量，解码器有一个开始符结束符.....	43

翻转输入数据 Reverse.....	43
4. 偷窥 Peeky.....	44
第九章 Attention	44
1. 对编码器的改进，hs（加权求和）.....	45
2. 模型 attention weight 是怎么实现的，attention sum 是什么意思，用hs 的每一个行点乘h 得到s， 再由softmax 函数得到权重	45

第一章 Numpy

1. 广播

```
>>> A = np.array([[1, 2], [3, 4]])
>>> B = np.array([10, 20])
>>> A * B
array([[ 10, 40],
       [ 30, 80]])
```



第二章 感知机

重点内容：

- 什么是感知机？
- 为什么要多层？
- 感知机的基本形式是什么？
- $w_1 w_2 b$ 是什么意思？
- 阶跃函数是什么意思？

1. 什么是感知机？

感知机接收多个输入信号，输出一个信号。

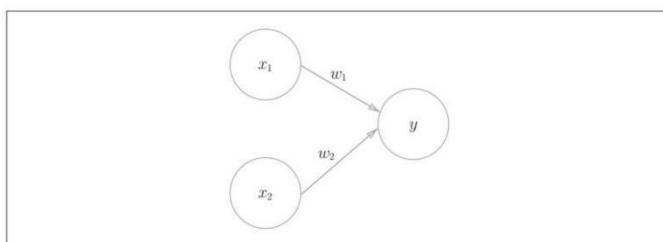


图 2-1 有两个输入的感知机

感知机的运行原理只有这些！把上述内容用数学式来表示，就是式 (2.1)。

$$y = \begin{cases} 0 & (w_1 x_1 + w_2 x_2 \leq \theta) \\ 1 & (w_1 x_1 + w_2 x_2 > \theta) \end{cases} \quad (2.1)$$

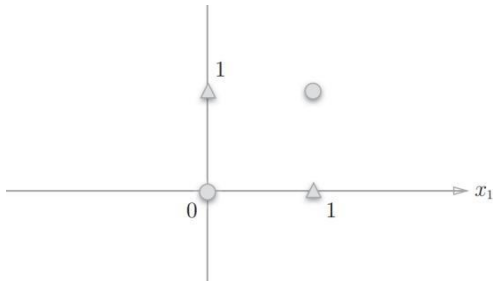
2. 导入权重与偏置

把式 (2.1) 的 θ 换成 $-b$ ， b 称为偏置， w_1 和 w_2 称为权重。偏置和权重 w_1 、 w_2 的作用是不一样的。

具体地说， w_1 和 w_2 是控制输入信号的重要性的参数，而偏置是调整神经元被激活的容易程度（输出信号为 1 的程度）的参数。

$$y = \begin{cases} 0 & (b + w_1 x_1 + w_2 x_2 \leq 0) \\ 1 & (b + w_1 x_1 + w_2 x_2 > 0) \end{cases} \quad (2.2)$$

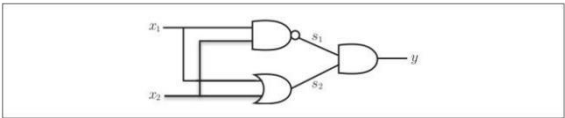
但是，感知机是无法实现异或门的，下面的图像是不能用一条直线区分开的。



3. 线性和非线性

由曲线分割而成的空间称为非线性空间，由直线分割而成的空间称为线性空间。

感知机不能表示异或门让人深感遗憾，但是感知机可以“叠加层”(通过叠加层来表示异或门是本节的要点)，两层叠加实现异或。叠加了多层的感知机也称为多层感知机 (multi-layered perceptron MLP)。



4. 阶跃函数

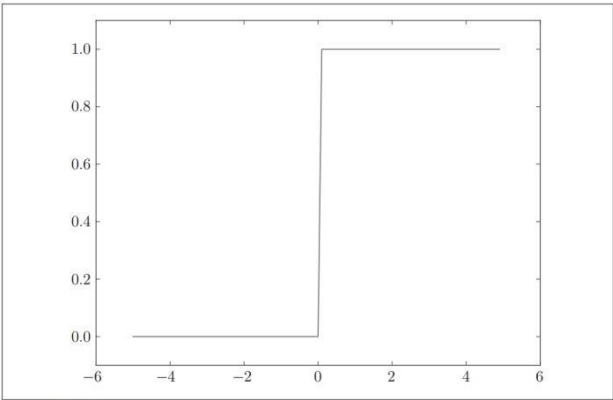


图 3-6 阶跃函数的图形

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases} \tag{3.3}$$

式 (3.3) 表示的激活函数以阈值为界，一旦输入超过阈值，就切换输出。这样的函数称为“阶跃函数”。

- 神经网络中的激活函数使用平滑变化的 sigmoid 函数或 ReLU 函数。
- 通过巧妙地使用 NumPy 多维数组，可以高效地实现神经网络。
- 机器学习的问题大体上可以分为回归问题和分类问题。
- 关于输出层的激活函数，回归问题中一般用恒等函数，分类问题中一般用 softmax 函数。
- 分类问题中，输出层的神经元的数量设置为要分类的类别数。
- 输入数据的集合称为批。通过以批为单位进行推理处理，能够实现高速的运算。

1. 什么是神经网络

神经网络的一个重要性质是它可以自动地从数据中学习合适的权重参数。

用图来表示神经网络的话，如图 3-1 所示。我们把最左边的一列称为输入层，最右边的一列称为输出层，中间的一列称为中间层。中间层有时也称为隐藏层。

输入层维度和数据相关，输出层和任务相关（分多少个类输出层就是多少），这两个维度一般

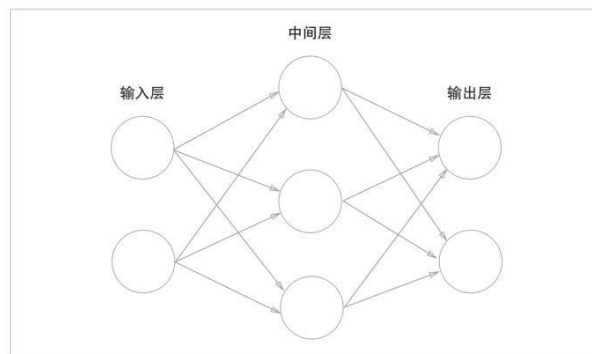


图 3-1 神经网络的例子

不变，中间层的数目会变。

$$y = h(b + w_1x_1 + w_2x_2)$$

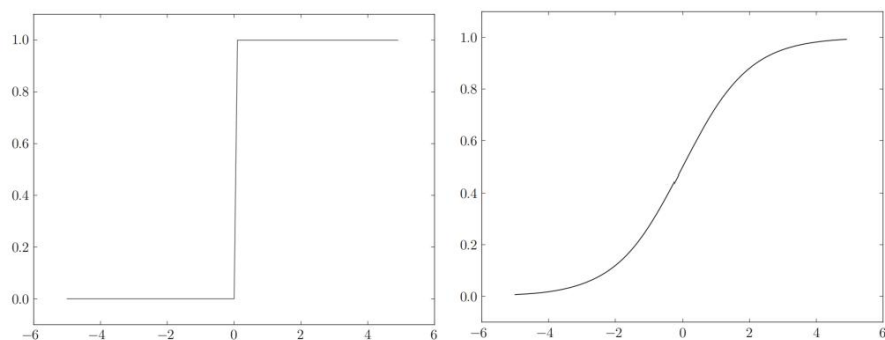
输入信号的总和会被函数 $h(x)$ 转换，转换后的值就是输出 y 。然后，式 (3.3) 所表示的函数 $h(x)$ ，在输入超过 0 时返回 1，否则返回 0。

2. 激活函数

刚才登场的 $h(x)$ 函数会将输入信号的总和转换为输出信号，这种函数一般称为激活函数 (activation function)。如“激活”一词所示，激活函数的作用在于决定如何来激活输入信号的总和。

sigmoid 函数：神经网络中经常使用的一个激活函数就是式 (3.6) 表示的 sigmoid 函数 (sigmoid function)。让阶跃函数连续并且尽可能相似。

$$h(x) = \frac{1}{1 + \exp(-x)} \quad (3.6)$$



可以发现它们具有相似的形状。实际上，两者的结构均是“输入小时，输出接近 0（为 0）；随着输入增大，输出向 1 靠近（变成 1）”。也就是说，当输入信号为重要信息时，阶跃函数和 sigmoid 函数都会输出较大的值；当输入信号为不重要的信息时，两者都输出较小的值。还有一个共同点是，不管输入信号有多小，或者有多大，输出信号的值都在 0 到 1 之间。

3. 非线性函数

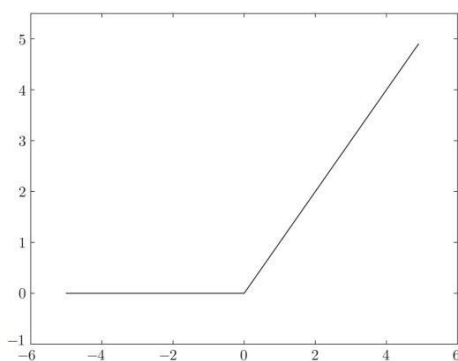
阶跃函数和 sigmoid 函数还有其他共同点，就是两者均为非线性函数。神经网络的激活函数必须使用非线性函数。换句话说，激活函数不能使用线性函数，使用线性函数的话，加深神经网络的层数就没有意义了。

4. ReLU (Rectified Linear Unit) 函数

这个函数更简单，可以减少计算量，一定程度解决梯度消失问题。

ReLU 函数可以表示为下面的式 (3.7)。

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases} \quad (3.7)$$



5. 神经网络的内积

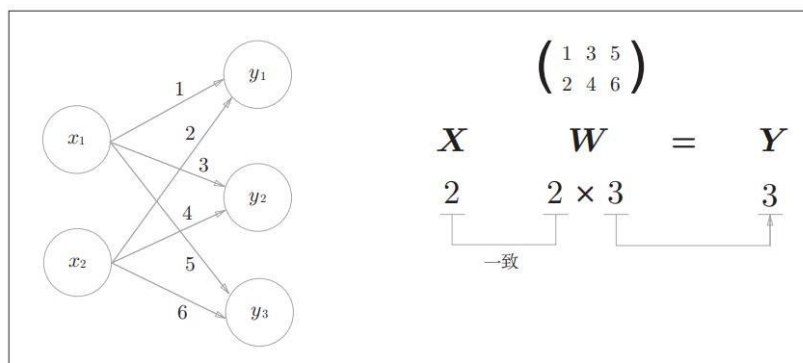
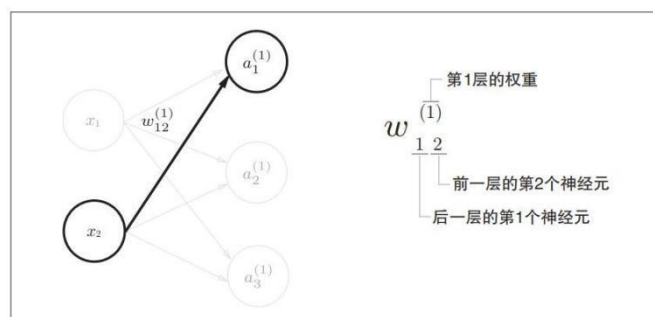


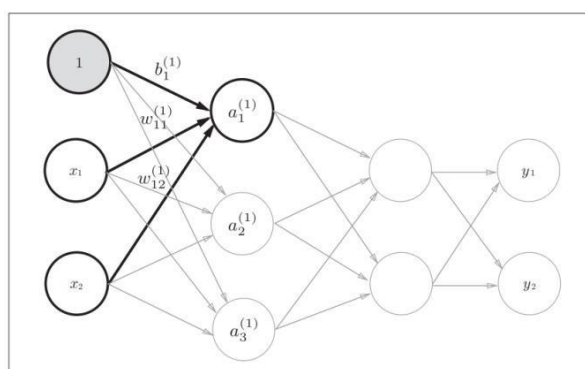
图 3-14 通过矩阵的乘积进行神经网络的运算

实现该神经网络时，要注意 $\mathbf{X}$ 、 $\mathbf{W}$ 、 $\mathbf{Y}$ 的形状，特别是 $\mathbf{X}$ 和 $\mathbf{W}$ 的对应维度的元素个数是否一致，这一点很重要。



权重符号

加上偏置符号：表示偏置的神经元“1”。请注意，偏置的右下角的索引号只有一个。这是因为前一层的偏置神经元（神经元“1”）只有一个



使用矩阵进行乘法运算：

$$\mathbf{A}^{(1)} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{B}^{(1)} \quad (3.9)$$

其中， $\mathbf{A}^{(1)}$ 、 $\mathbf{X}$ 、 $\mathbf{B}^{(1)}$ 、 $\mathbf{W}^{(1)}$ 如下所示。

$$\mathbf{A}^{(1)} = \begin{pmatrix} a_1^{(1)} & a_2^{(1)} & a_3^{(1)} \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} x_1 & x_2 \end{pmatrix}, \quad \mathbf{B}^{(1)} = \begin{pmatrix} b_1^{(1)} & b_2^{(1)} & b_3^{(1)} \end{pmatrix}$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix}$$

隐藏层的加权和（加权信号和偏置的总和）用 a 表示，被激活函数转换后的信号用 z 表示。此外，图

中 $h()$ 表示激活函数，这里我们使用的是 sigmoid 函数

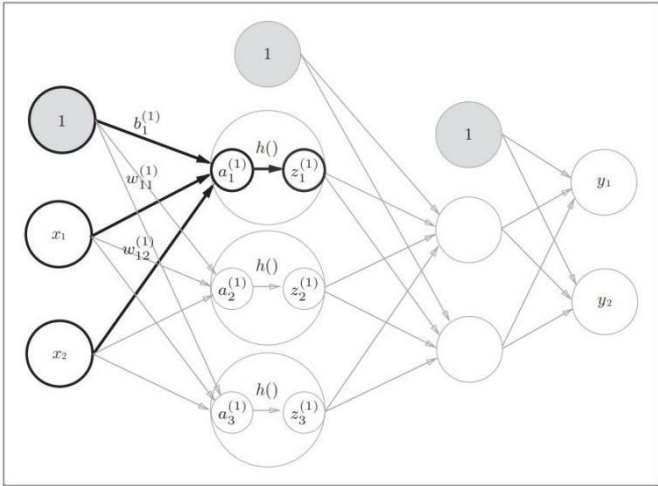
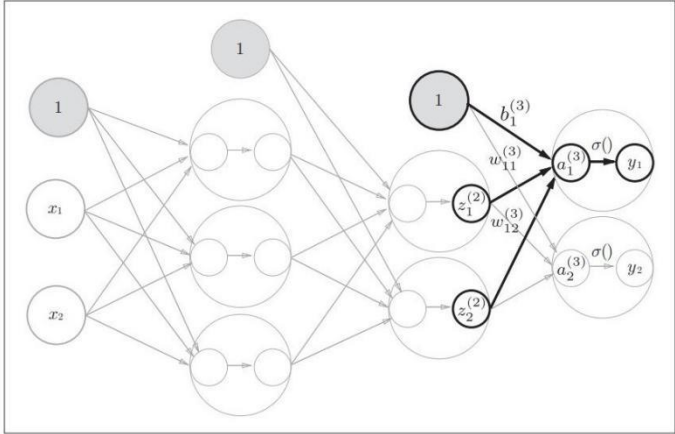


图 3-18 从输入层到第 1 层的信号传递

输出层：



一般地，回归问题可以使用恒等函数，二元分类问题可以使用 sigmoid 函数，多元分类问题可以使用 softmax 函数。

forward（前向）：它表示的是从输入到输出方向的传递处理。

backward：从输出到输入方向的处理

分类问题中使用的 softmax 函数可以用下面的式 (3.10) 表示。

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)} \quad (3.10)$$

6. softmax 函数：记住公式，实现归一，使输出结果加合为 1

softmax 函数的改进：这样做的好处是，现原本为 nan（not a number，不确定）的地方，现在被正确计算了。

$$\begin{aligned}
 y_k &= \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)} = \frac{C \exp(a_k)}{C \sum_{i=1}^n \exp(a_i)} \\
 &= \frac{\exp(a_k + \log C)}{\sum_{i=1}^n \exp(a_i + \log C)} \\
 &= \frac{\exp(a_k + C')}{\sum_{i=1}^n \exp(a_i + C')}
 \end{aligned} \tag{3.11}$$

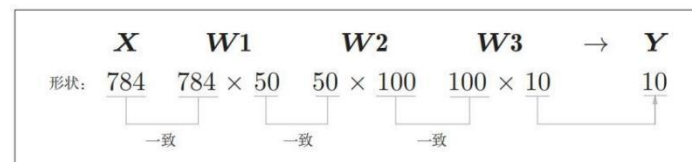
一般而言，神经网络只把输出值最大的神经元所对应的类别作为识别结果。并且，即便使用 softmax 函数，输出值最大的神经元的位置也不会变。

求解机器学习问题的步骤可以分为“学习”和“推理”两个阶段。首先，在学习阶段进行模型的学习，然后，在推理阶段，用学到的模型对未知的数据进行推理（分类）。如前所述，推理阶段一般会省略输出层的 softmax 函数。

把数据限定到某个范围内的处理称为正规化（normalization）。此外，对神经网络的输入数据进行某种既定的转换称为预处理（pre-processing）。这里，作为对输入图像的一种预处理，我们进行了正规化

7. 批处理

使用批处理前：



使用批处理后：

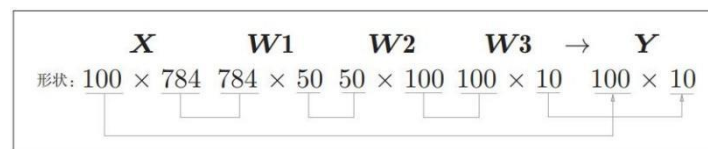


图 3-27 批处理中数组形状的变化

表示输入的 100 张图像的结果被一次性输出了。这种打包式的输入数据称为批（batch）。

批处理对计算机的运算大有利处，可以大幅缩短每张图像的处理时间。这是因为大多数处理数值计算的库都进行了能够高效处理大型数组运算的最优化。并且，在神经网络的运算中，当数据传送成为瓶颈时，批处理可以减轻数据总线的负荷（严格地讲，相对于数据读入，可以将更多的时间用在计算上）。也就是说，批处理一次性计算大型数组要比分开逐步计算各个小型数组速度更快。

重点内容：

- 几种损失函数
- 为什么要设置损失函数

- 什么是数值微分
- 什么是随机梯度下降

神经网络的特征就是可以从数据中学习。

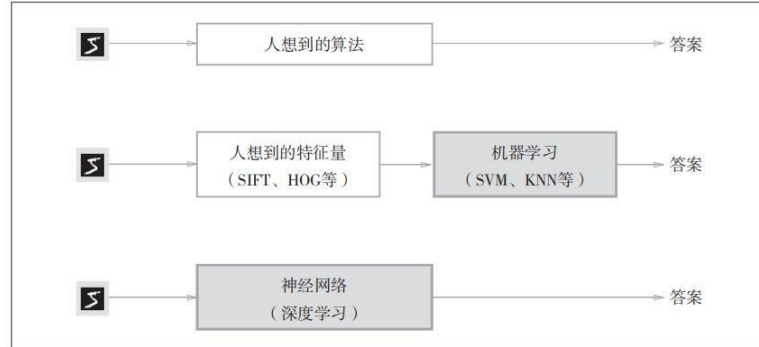


图 4-2 从人工设计规则转变为由机器从数据中学习：没有人为介入的方块用灰色表示

机器学习中，一般将数据分为训练数据和测试数据两部分来进行学习和实验等。为了正确评价模型的泛化能力，就必须划分训练数据和测试数据。另外，训练数据也可以称为监督数据。

泛化能力是指处理未被观察过的数据（不包含在训练数据中的数据）的能力。获得泛化能力是机器学习的最终目标。只对某个数据集过度拟合的状态称为过拟合（over fitting）。

神经网络的学习中所用的指标称为损失函数（loss function）。这个损失函数可以使用任意函数，但一般用均方误差和交叉熵误差等。

1. 均方误差

可以用作损失函数的函数有很多，其中最有名的是均方误差（mean squared error）。均方误差如下式所示。

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2 \quad (4.1)$$

2. one-hot 编码

```
>>> y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]
>>> t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
```

其中第二个数据的概率最大，因此在 y 中将第二个数据设为 1。将正确解标签表示为 1，其他标签表示为 0 的表示方法称为 one-hot 表示。



3. 交叉熵误差

除了均方误差之外，交叉熵误差（cross entropy error）也经常被用作损失函数。交叉熵误差如下式所示。

$$E = - \sum_k t_k \log y_k \quad (4.2)$$

加上了一个微小值 delta。这是因为，当出现 np.log(0) 时，np.log(0) 会变为负无限大的 -inf，这样一来就会导致后续计算无法进行。作为保护性对策，添加一个微小值可以防止负无限大的发生。

$$E = -\frac{1}{N} \sum_n \sum_k t_{nk} \log y_{nk} \tag{4.3}$$

假设数据有 N 个， tnk 表示第 n 个数据的第 k 个元素的值（ ynk 是神经网络的输出， tnk 是监督数据）。只是把求单个数据的损失函数的式扩大到了 N 份数据。

4. mini-batch

神经网络的学习也是从训练数据中选出一批数据（称为 mini-batch, 小批量），然后对每个 mini-batch 进行学习。比如，从 60000 个训练数据中随机选择 100 笔，再用这 100 笔数据进行学习。这种学习方式称为 mini-batch 学习。

由于 one-hot 表示中 t 为 0 的元素的交叉熵误差也为 0，因此针对这些元素的计算可以忽略。换言之，如果可以获得神经网络在正确解标签处的输出，就可以计算交叉熵误差。

5. 为什么要设置损失函数？

在进行神经网络的学习时，**不能将识别精度作为指标**。因为如果以识别精度为指标，则参数的导数在绝大多数地方都会变为 0。

在神经网络的学习中，寻找最优参数（权重和偏置）时，要寻找使损失函数的值尽可能小的参数。为了找到使损失函数的值尽可能小的地方，需要计算参数的导数（确切地讲是梯度），然后以这个导数为指引，逐步更新参数的值。

阶跃函数就像“竹筒敲石”一样，只在某个瞬间产生变化。而 sigmoid 函数，不仅函数的输出（竖轴的值）是连续变化的，曲线的斜率（导数）也是连续变化的。也就是说，sigmoid 函数的导数在任何地方都不为 0。这对神经网络的学习非常重要。得益于这个斜率不会为 0 的性质，神经网络的学习得以正确进行。

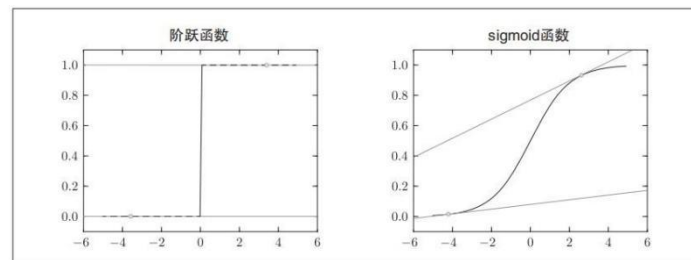


图 4-4 阶跃函数和 sigmoid 函数：阶跃函数的斜率在绝大多数地方都为 0，而 sigmoid 函数的斜率（切线）不会为 0

6. 数值微分

利用微小的差分求导数的过程称为数值微分（numerical differentiation）。而基于数学式的推导求导数的过程，则用“解析性”（analytic）一词，称为“解析性求解”或者“解析性求导”。

偏导数：

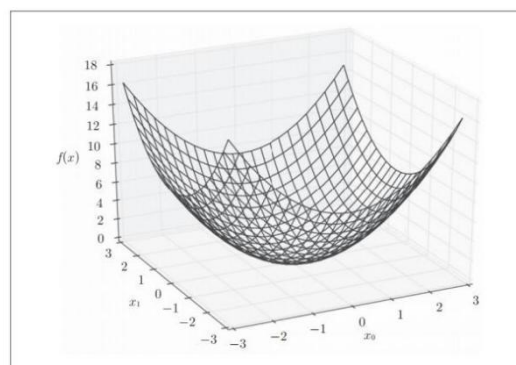


图 4-8 $f(x_0, x_1) = x_0^2 + x_1^2$ 的图像

现在我们来求式(4.6)的导数。这里需要注意的是，式(4.6)有两个变量，所以有必要区分对哪个变量求导数，即对 x_0 和 x_1 两个变量中的哪一个求导数。另外，我们把这里讨论的有多个变量的函数的导数称为偏导数。用数学式表示的话，可以写成 $\frac{\partial f}{\partial x_0}$ 、 $\frac{\partial f}{\partial x_1}$ 。

偏导数和单变量的导数一样，都是求某个地方的斜率。不过，偏导数需要将多个变量中的某一个变量定为目标变量，并将其他变量固定为某个值。在上例的代码中，为了将目标变量以外的变量固定到某些特定的值上，我们定义了新函数。然后，对新定义的函数应用了之前的求数值微分的函数，得到偏导数。

7. 梯度

的偏导数 $(\frac{\partial f}{\partial x_0}, \frac{\partial f}{\partial x_1})$ 。另外，像 $(\frac{\partial f}{\partial x_0}, \frac{\partial f}{\partial x_1})$ 这样的由全部变量的偏导数汇总而成的向量称为梯度 (gradient)。梯度可以像下面这样来实现。

如图 4-9 所示， $f(x_0, x_1) = x_0^2 + x_1^2$ 的梯度呈现为有向向量(箭头)。观察图 4-9，我们发现梯度指向函数 $f(x_0, x_1)$ 的“最低处”(最小值)，就像指南针一样，所有的箭头都指向同一点。其次，我们发现离“最低处”越远，箭头越大。

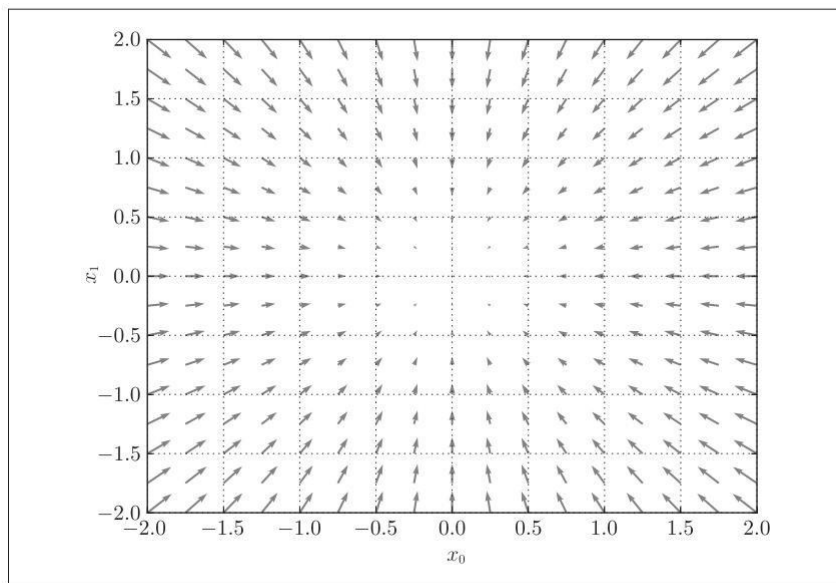


图4-9 $f(x_0, x_1) = x_0^2 + x_1^2$ 的梯度

函数的极小值、最小值以及被称为鞍点 (saddle point) 的地方，梯度为 0。在梯度法中，函数的取值从当前位置沿着梯度方向前进一定距离，然后在新的地方重新求梯度，再沿着新梯度方向前进，如此反复，不断地沿梯度方向前进。像这样，通过不断地沿梯度方向前进，逐渐减小函数值的过程就是梯度法 (gradient method)。

$$\begin{aligned} x_0 &= x_0 - \eta \frac{\partial f}{\partial x_0} \\ x_1 &= x_1 - \eta \frac{\partial f}{\partial x_1} \end{aligned} \quad (4.7)$$

式 (4.7) 的 η 表示更新量，在神经网络的学习中，称为学习率 (learning rate)。学习率决定在一次学习中，应该学习多少，以及在多大程度上更新参数。

像学习率这样的参数称为超参数。这是一种和神经网络的参数 (权重和偏置) 性质不同的参数。相对

于神经网络的权重参数是通过训练数据和学习算法自动获得的，学习率这样的超参数则是人工设定的。

神经网络的梯度：

$$\mathbf{W} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix}$$
$$\frac{\partial L}{\partial \mathbf{W}} = \begin{pmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{12}} & \frac{\partial L}{\partial w_{13}} \\ \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{22}} & \frac{\partial L}{\partial w_{23}} \end{pmatrix} \quad (4.8)$$

8. 学习算法的实现

- ❶ 前提：神经网络存在合适的权重和偏置，调整权重和偏置以便拟合训练数据的过程称为“学习”。神经网络的学习分成下面 4 个步骤。
- ❷ 步骤 1（mini-batch）：从训练数据中随机选出一部分数据，这部分数据称为 mini-batch。我们的目标是减小 mini-batch 的损失函数的值。
- ❸ 步骤 2（计算梯度）：为了减小 mini-batch 的损失函数的值，需要求出各个权重参数的梯度。梯度表示损失函数的值减小最多的方向。
- ❹ 步骤 3（更新参数）：将权重参数沿梯度方向进行微小更新。
- ❺ 步骤 4（重复）：重复步骤 1、步骤 2、步骤 3。

神经网络的学习按照上面 4 个步骤进行。这个方法通过梯度下降法更新参数，不过因为这里使用的数据是随机选择的 mini batch 数据，所以又称为随机梯度下降法（stochastic gradient descent）。“随机”指的是“随机选择的”的意思，因此，随机梯度下降法是“对随机选择的数据进行的梯度下降法”。

深度学习的很多框架中，随机梯度下降法一般由一个名为 SGD 的函数来实现。SGD 来源于随机梯度下降法的英文名称的首字母

9. 基于测试数据的评价

epoch 是一个单位。一个 epoch 表示学习中所有训练数据均被使用过一次时的更新次数。比如，对于 10000 笔训练数据，用大小为 100 笔数据的 mini-batch 进行学习时，重复随机梯度下降法 100 次，所有的训练数据就都被“看过”了。此时，100 次就是一个 epoch。

第五章 误差反向传播算法

重点内容：

- 掌握计算图反向传播怎么画

一个能够高效计算权重参数的梯度的方法——误差反向传播法

要正确理解误差反向传播法，两种方法：一种是基于数学式；另一种是基于计算图（computational graph）。

1. 计算图法

“从左向右进行计算”是一种正方向上的传播，简称为正向传播（forward propagation）。正向传播是从计算图出发点 to 结束点的传播。既然有正向传播这个名称，当然也可以考虑反向（从图上看的话，就是从右向左）的传播。

计算图的特征是可以通过传递“局部计算”获得最终结果。“局部”这个词的意思是“与自己相关的某个小范围”。使用计算图最大的原因是，可以通过反向传播高效计算导数。

链式法则

原理：如果某个函数由复合函数表示，则该复合函数的导数可以用构成复合函数的各个函数的导数的乘积表示。

加法的反向传播：

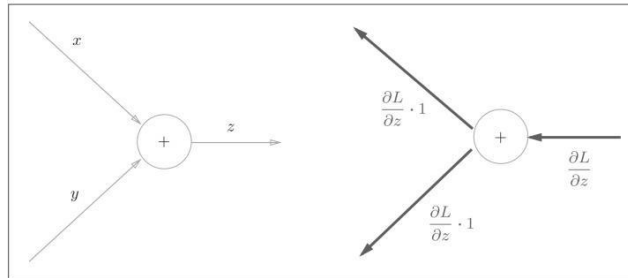


图 5-9 加法节点的反向传播：左图是正向传播，右图是反向传播。如右图的反向传播所示，加法节点的反向传播将上游的值原封不动地输出到下游

乘法的反向传播：

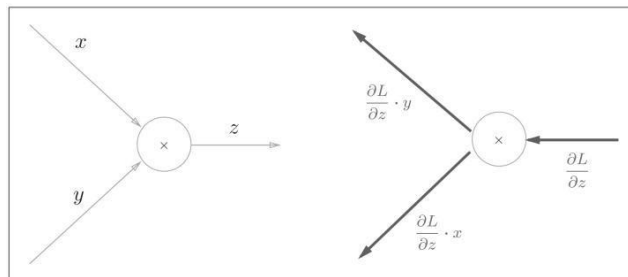


图 5-12 乘法的反向传播：左图是正向传播，右图是反向传播

举个栗子：

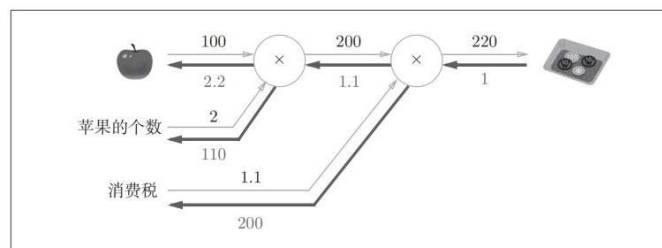


图 5-14 购买苹果的反向传播的例子

下面这个图要求一定掌握

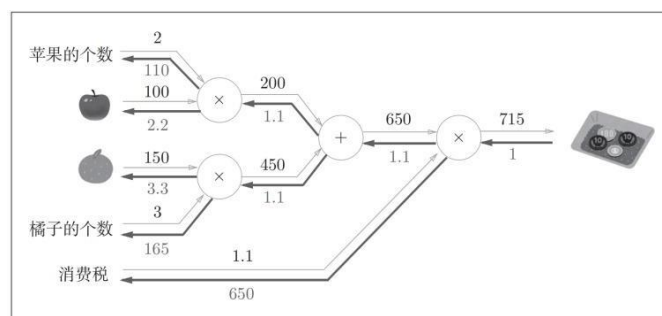


图 5-17 购买2个苹果和3个橘子

2. 乘法层的实现

层的实现中有两个共通的方法（接口）`forward()`和`backward()`。`forward()`对应正向传播，`backward()`对

应反向传播。

__init__()中会初始化实例变量x 和 y，它们用于保存正向传播时的输入值。forward()接收 x 和 y 两个参数，将它们相乘后输出。backward()将从上游传来的导数（dout）乘以正向传播的翻转值，然后传给下游。

3. Relu 实现

激活函数 ReLU (Rectified Linear Unit) 由下式 (5.7) 表示。

$$y = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases} \quad (5.7)$$

通过式 (5.7)，可以求出 y 关于 x 的导数，如式 (5.8) 所示。

$$\frac{\partial y}{\partial x} = \begin{cases} 1 & (x > 0) \\ 0 & (x \leq 0) \end{cases} \quad (5.8)$$

如果正向传播时的输入 x 大于 0，则反向传播会将上游的值原封不动地传给下游。反过来，如果正向传播时的 x 小于等于 0，则反向传播中传给下游的信号将停在此处。

ReLU 层的作用就像电路中的开关一样。正向传播时，有电流通过的话，就将开关设为 ON；没有电流通过的话，就将开关设为 OFF。反向传播时，开关为 ON 的话，电流会直接通过；开关为 OFF 的话，则不会有电流通过。

4. Sigmoid 实现

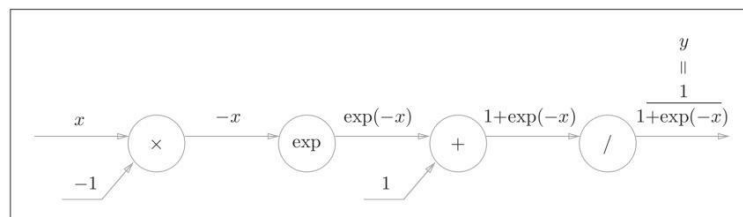


图 5-19 sigmoid层的计算图(仅正向传播)

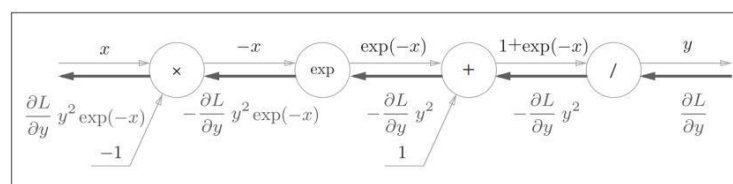


图 5-20 Sigmoid层的计算图

另外， $\frac{\partial L}{\partial y} y^2 \exp(-x)$ 可以进一步整理如下。

$$\begin{aligned} \frac{\partial L}{\partial y} y^2 \exp(-x) &= \frac{\partial L}{\partial y} \frac{1}{(1 + \exp(-x))^2} \exp(-x) \\ &= \frac{\partial L}{\partial y} \frac{1}{1 + \exp(-x)} \frac{\exp(-x)}{1 + \exp(-x)} \\ &= \frac{\partial L}{\partial y} y(1 - y) \end{aligned} \quad (5.12)$$

5. Affine/Softmax 实现

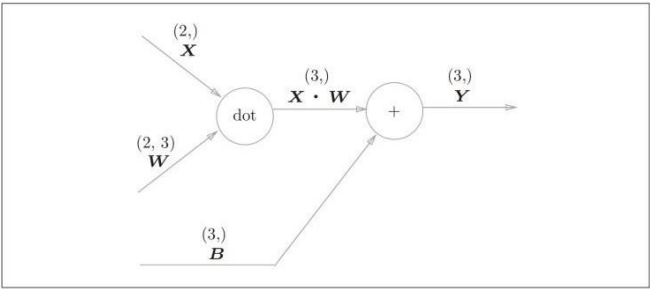


图 5-24 Affine层的计算图(注意变量是矩阵, 各个变量的上方标记了该变量的形状)

X、W、B 是矩阵 (多维数组)。之前我们见到的计算图中各个节点间流动的是标量, 而这个例子中各

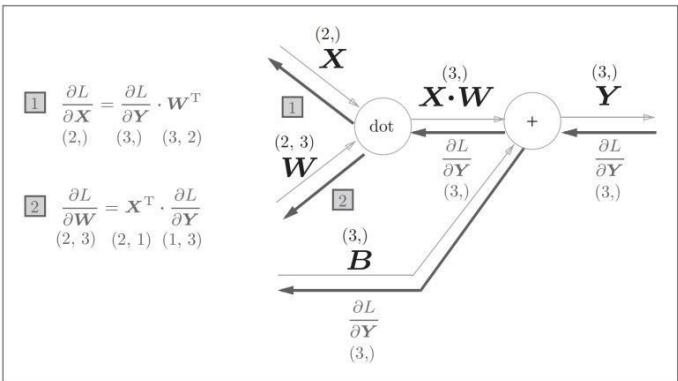


图 5-25 Affine层的反向传播: 注意变量是多维数组。反向传播时各个变量的下方标记了该变量的形状

个节点间传播的是矩阵

批版本的 Affine

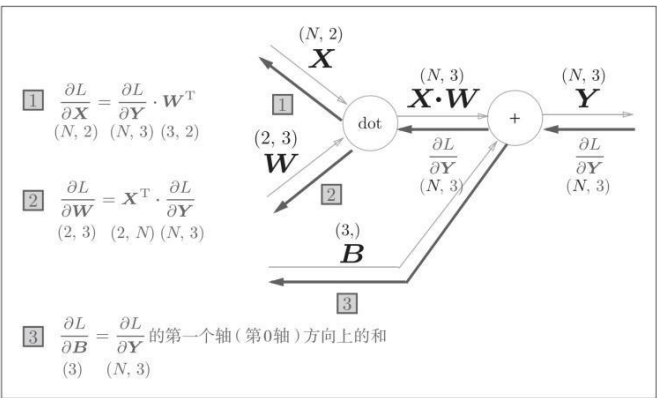


图 5-27 批版本的 Affine 层的计算图

6. softmax-with-loss

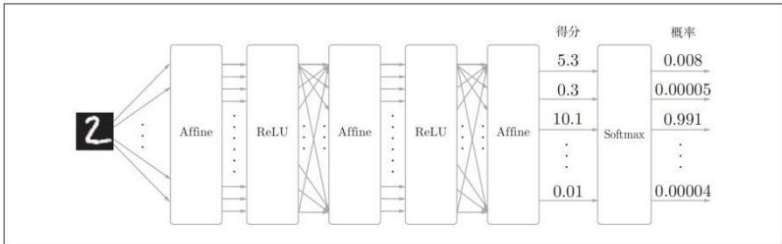


图 5-28 输入图像通过 Affine 层和 ReLU 层进行转换, 10 个输入通过 Softmax 层进行正规化。在这个例子中, “0” 的得分是 5.3, 这个值经过 Softmax 层转换为 0.008 (0.8%); “2” 的得分是 10.1, 被转换为 0.991 (99.1%)

下面来实现 Softmax 层。考虑到这里也包含作为损失函数的交叉熵误差（cross entropy error），所以称为“Softmax-with-Loss 层”。Softmax-with-Loss 层（Softmax 函数和交叉熵误差）的计算图如图所示

理解一下，加法怎么做，乘法怎么做

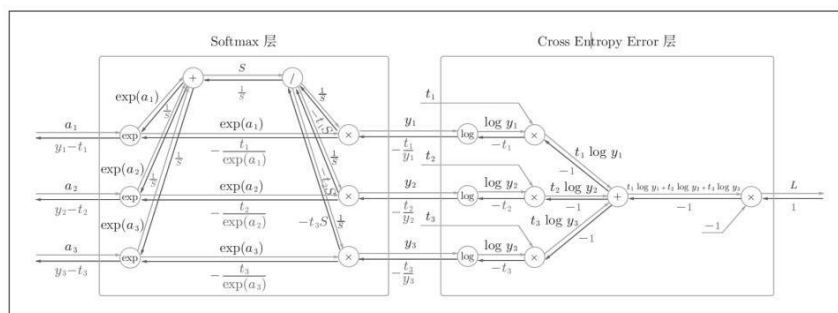


图5-29 Softmax-with-Loss 层的计算图

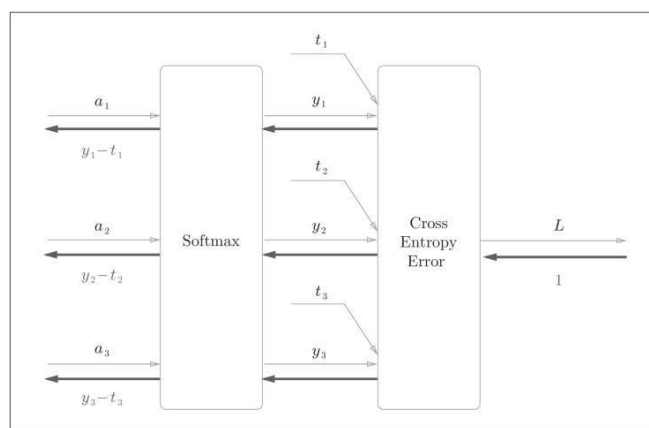


图5-30 “简易版”的 Softmax-with-Loss 层的计算图

化简后：

由于 (y_1, y_2, y_3) 是 Softmax 层的输出， (t_1, t_2, t_3) 是监督数据，所以 $(y_1 - t_1, y_2 - t_2, y_3 - t_3)$ 是 Softmax 层的输出和监督标签的差分，直截了当地表示了当前神经网络的输出与监督标签的误差。神经网络的反向传播会把这个差分表示的误差传递给前面的层，这是神经网络学习中的重要性质。

7. 误差反向传播算法的实现

数值微分的作用：数值微分的优点是实现简单，因此，一般情况下不太容易出错。而误差反向传播法的实现很复杂，容易出错。所以，经常会比较数值微分的结果和误差反向传播法的结果，以确认误差反向传播法的实现是否正确。确认数值微分求出的梯度结果和误差反向传播法求出的结果是否一致（严格地讲，是非常相近）的操作称为梯度确认（gradient check）。

第六章 神经网络的学习技巧

重点内容：

- 什么是随机梯度下降
- 几种参数更新方法：SGD、Momentum、AdaGrad、Adam
- 权重初始化的方法：Xavier、He

- 什么是 Batch Normalization
- 什么是过拟合，抑制过拟合的方法：正则化、权值衰减、Dropout
- 验证集是什么，作用：调整超参数

1. 参数更新

寻找最优参数的问题，解决这个问题的过程称为**最优化（optimization）**。

随机梯度下降法：使用参数的梯度，沿梯度方向更新参数，并重复这个步骤多次，从而逐渐靠近最优参数，这个过程称为随机梯度下降法（stochastic gradient descent），简称 SGD。**公式也要会**

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}} \quad (6.1)$$

这里把需要更新的权重参数记为 $\mathbf{W}$ ，把损失函数关于 $\mathbf{W}$ 的梯度记为 $\frac{\partial L}{\partial \mathbf{W}}$ 。 η 表示学习率，实际上会取 0.01 或 0.001 这些事先决定好的值。式子中的 $\leftarrow$ 表示用右边的值更新左边的值。

SGD 的缺点：虽然 SGD 简单，并且容易实现，但是在解决某些问题时可能没有效率。SGD 的缺点是，如果函数的形状非均向（anisotropic），比如呈延伸状，搜索的路径就会非常低效。因此，我们需要比单纯朝梯度方向前进的 SGD 更聪明的方法。SGD 低效的根本原因是，梯度的方向并没有指向最小值的方向。

知道随机梯度下降会遇到什么问题！

2. Momentum

Momentum 是“动量”的意思，和物理有关。用数学式表示 Momentum 方法，如下所示。

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \eta \frac{\partial L}{\partial \mathbf{W}} \quad (6.3)$$

$$\mathbf{W} \leftarrow \mathbf{W} + \mathbf{v} \quad (6.4)$$

3. AdaGrad

在神经网络的学习中，学习率（数学式中记为 η ）的值很重要。学习率过小，会导致学习花费过多时间；反过来，学习率过大，则会导致学习发散而不能正确进行。在关于学习率的有效技巧中，有一种被称为学习率衰减（learning rate decay）的方法，**即随着学习的进行，使学习率逐渐减小**。实际上，一开始“多”学，然后逐渐“少”学的方法，在神经网络的学习中经常被使用。

$$\mathbf{h} \leftarrow \mathbf{h} + \frac{\partial L}{\partial \mathbf{W}} \odot \frac{\partial L}{\partial \mathbf{W}} \quad (6.5)$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{1}{\sqrt{\mathbf{h}}} \frac{\partial L}{\partial \mathbf{W}} \quad (6.6)$$

这里新出现了变量 $\mathbf{h}$ ，如式(6.5)所示，它保存了以前的所有梯度值的平方和。然后，在更新参数时，通过乘以 $\frac{1}{\sqrt{\mathbf{h}}}$ ，就可以调整学习的尺度。这意味着，参数的元素中变动较大（被大幅更新）的元素的学习率将变小。也就是说，可以按参数的元素进行学习率衰减，使变动大的参数的学习率逐渐减小。

AdaGrad 会记录过去所有梯度的平方和。因此，学习越深入，更新的幅度就越小。实际上，如果无止境地学习，更新量就会变为 0，完全不再更新。为了改善这个问题，可以使用 RMSProp 方法。RMSProp 方

法并不是将过去所有的梯度一视同仁地相加，而是逐渐地遗忘过去的梯度，在做加法运算时将新梯度的信息更多地反映出来。

4. Adam

Momentum 参照小球在碗中滚动的物理规则进行移动，AdaGrad 为参数的每个元素适当地调整更新步伐。将这两个方法融合在一起。

Adam 会设置 3 个超参数。一个是学习率（论文中以 α 出现），另外两个是一次 momentum 系数 β_1 和二次 momentum 系数 β_2 。根据论文，标准的设定值是 β_1 为 0.9， β_2 为 0.999。设置了这些值后，大多数情况下都能顺利运行。

四种方法的比较：

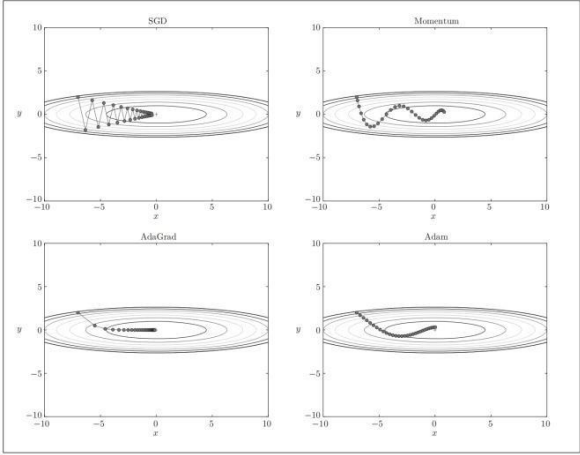


图 6-8 最优化方法的比较：SGD、Momentum、AdaGrad、Adam

5. 权重的初始值

为什么不能将权重初始值设为 0 呢？严格地说，为什么不能将权重初始值设成一样的值呢？这是因为在误差反向传播法中，所有的权重值都会进行相同的更新。

为什么初始值小一点更好？

观察权重初始值是如何影响隐藏层的激活值的分布的。这里要做的实验是，向一个 5 层神经网络（激

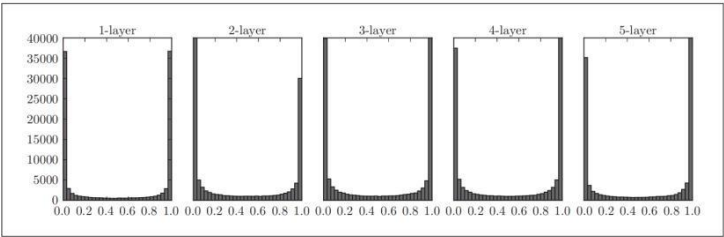


图 6-10 使用标准差为 1 的高斯分布作为权重初始值时的各层激活值的分布

活函数使用 sigmoid 函数）传入随机生成的输入数据，用直方图绘制各层激活值的数据分布。

各层的激活值呈偏向 0 和 1 的分布。这里使用的 sigmoid 函数是 S 型函数，随着输出不断地靠近 0（或者靠近 1），它的导数的值逐渐接近 0。因此，偏向 0 和 1 的数据分布会造成反向传播中梯度的值不断变小，最后消失。这个问题称为梯度消失（gradient vanishing）。层次加深时，梯度消失的问题更严重。

下面，将权重的标准差设为 0.01，进行相同的实验。没有梯度消失问题。

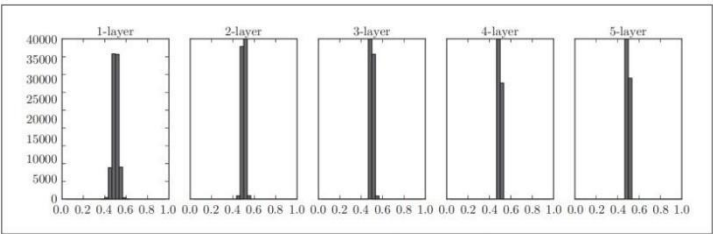


图 6-11 使用标准差为 0.01 的高斯分布作为权重初始值时的各层激活值的分布

因为如果有多个神经元都输出几乎相同的值，那它们就没有存在的意义了。比如，如果 100 个神经元都输出几乎相同的值，那么也可以由 1 个神经元来表达基本相同的事情。

现在，在一般的深度学习框架中，Xavier 初始值已被作为标准使用。比如，Caffe 框架中，通过在设定权重初始值时赋予 xavier 参数，就可以使用 Xavier 初始值。Xavier 的论文中，为了使各层的激活值呈现出具有相同广度的分布，推导出了合适的权重尺度。推导出的结论是，如果前一层的节点数为 n ，则初始值使用标

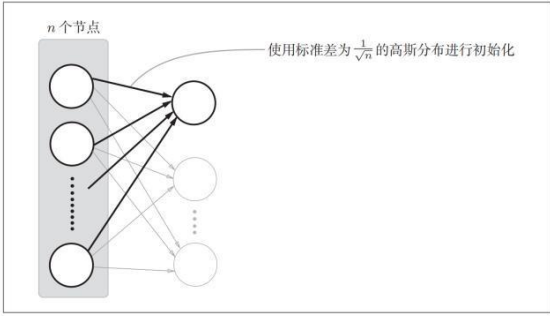


图 6-12 Xavier 初始值：与前一层有 n 个节点连接时，初始值使用标准差为 $\frac{1}{\sqrt{n}}$ 的分布

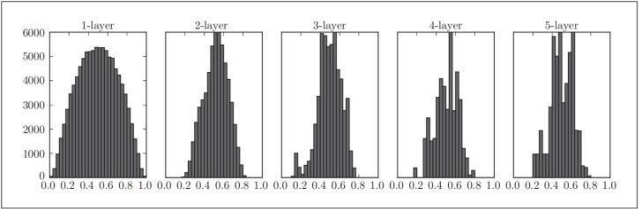


图 6-13 使用 Xavier 初始值作为权重初始值时的各层激活值的分布

准差为 $1/n^{(1/2)}$ 的分布

6. Relu 初始值

Xavier 初始值是以激活函数是线性函数为前提而推导出来的。因为 sigmoid 函数和 tanh 函数左右对称，且中央附近可以视作线性函数，所以适合使用 Xavier 初始值。但当激活函数使用 ReLU 时，一般推荐使用 ReLU 专用的初始值，也就是 Kaiming He 等人推荐的初始值，也称为“He 初始值”。

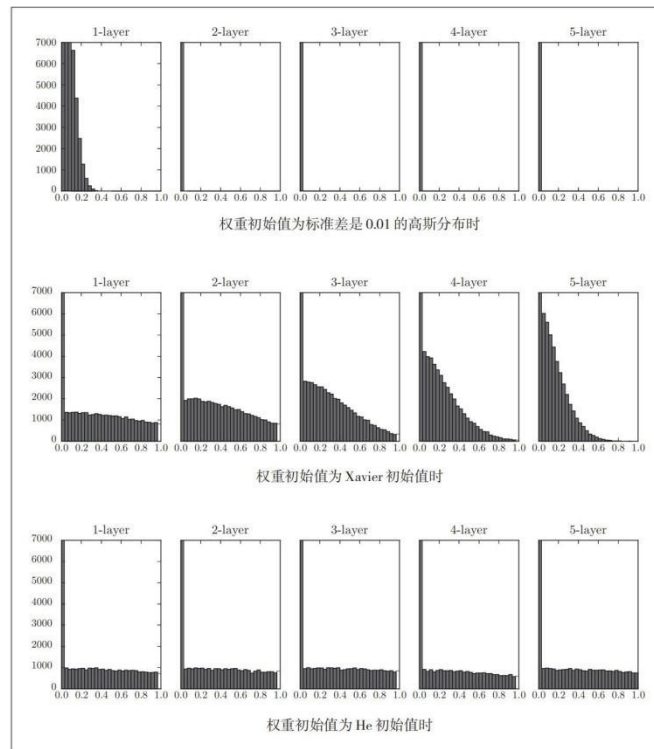


图 6-14 激活函数使用 ReLU 时,不同权重初始值的激活值分布的变化

接下来是初始值为 Xavier 初始值时的结果。在这种情况下，随着层的加深，偏向一点点变大。实际上，层加深后，激活值的偏向变大，学习时会出现梯度消失的问题。而当初始值为 He 初始值时，各层中分布的广度相同。由于即便层加深，数据的广度也能保持不变，因此逆向传播时，也会传递合适的值。

7. Batch Normalization

为了使各层拥有适当的广度，“强制性”地调整激活值的分布

Batch Norm 的优点：

- (1) 可以使学习快速进行（可以增大学习率）
- (2) 不那么依赖初始值（对于初始值不用那么神经质）
- (3) 抑制过拟合（降低 Dropout 等的必要性）

Batch Norm 的思路是调整各层的激活值分布使其拥有适当的广度。为此，要向神经网络中插入对数据

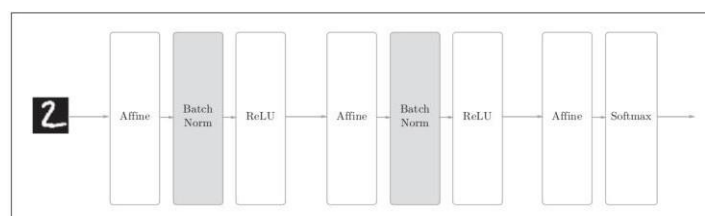


图 6-16 使用了 Batch Normalization 的神经网络的例子 (Batch Norm 层的背景为灰色)

分布进行正规化的层，即 Batch Normalization 层（下文简称 Batch Norm 层）

Batch Norm，顾名思义，以进行学习时的 mini-batch 为单位，按 mini-batch 进行正规化。具体而言，就是进行使数据分布的均值为 0、方差为 1 的正规化。用数学式表示的话，如下所示

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

将 mini-batch 的输入数据 {x1, x2, ..., xm} 变换为均值为 0、方差为 1 的数据，通过把这个处理插入到激活函数的前面（或者后面），可以减小数据分布的偏向。接着，Batch Norm 层会对正规化后的数据进行缩

$$y_i \leftarrow \gamma \hat{x}_i + \beta$$

放和平移的变换，用数学式可以如下表示：

这里， γ 和 β 是参数。一开始 $\gamma = 1$ ， $\beta = 0$ ，然后再通过学习调整到合适的值。

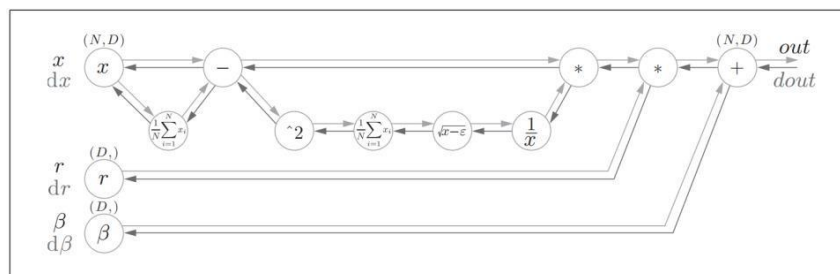


图6-17 Batch Normalization的计算图(引用自文献[13])

几乎所有的情况下都是使用 Batch Norm 时学习进行得更快

8. 正则化

过拟合指的是只能拟合训练数据，但不能很好地拟合不包含在训练数据中的其他数据的状态

发生过拟合的原因：

- (1) 模型拥有大量参数、表现力强
- (2) 训练数据少

正则化思想：限制模型参数的表达

抑制过拟合的方法：

- (3) **权值衰减：一直以来经常被使用的一种抑制过拟合的方法。该方法通过在学习的过程中对大的权重进行惩罚，来抑制过拟合。很多过拟合原本就是因为权重参数取值过大才发生的。**
- (4) **Dropout：Dropout 是一种在学习的过程中随机删除神经元的方法**

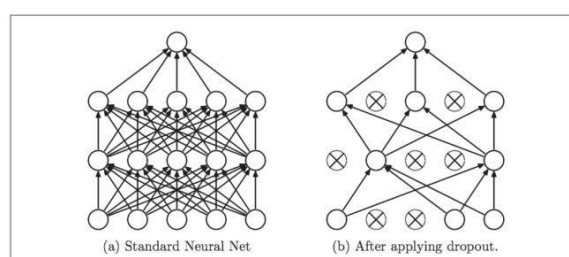


图6-22 Dropout的概念图(引用自文献[14])：左边是一般的神经网络，右边是应用了Dropout的网络。Dropout通过随机选择并删除神经元，停止向前传递信号

9. 超参数验证

超参数：各层的神经元数量、batch 大小、参数更新时的学习率或权值衰减等。

面我们要对超参数设置各种各样的值以进行验证。这里要注意的是，**不能使用测试数据评估超参数的性能**。

为什么不能用测试数据评估超参数的性能呢？这是因为如果使用测试数据调整超参数，**超参数的值会对测试数据发生过拟合**。换句话说，**用测试数据确认超参数的值的“好坏”，就会导致超参数的值被调整为只拟合测试数据**。这样的话，**可能就会得到不能拟合其他数据、泛化能力低的模型**。

因此，调整超参数时，**必须使用超参数专用的确认数据**。**用于调整超参数的数据，一般称为验证数据 (validation data)**。我们使用这个验证数据来评估超参数的好坏。

超参数最优化的过程

1. 设定超参数的范围。
2. 从设定的超参数范围中随机采样。
3. 使用步骤 1 中采样到的超参数的值进行学习，**通过验证数据评估识别精度（但是要將 epoch 设置得很小）**。
4. 重复步骤 1 和步骤 2（100 次等），**根据它们的识别精度的结果，缩小超参数的范围**。

第七章 卷积神经网络

重点内容：

- **什么是全连接层**
- **CNN 的结构**
- **卷积、padding、stride 的含义**
- **根据输入尺寸、滤波器尺寸计算输出尺寸**
- **什么是池化**
 - CNN 在此前的全连接层的网络中新增了卷积层和池化层。
 - 使用 im2col 函数可以简单、高效地实现卷积层和池化层。
 - 通过 CNN 的可视化，可知随着层次变深，提取的信息愈加高级。
 - LeNet 和 AlexNet 是 CNN 的代表性网络。
 - 在深度学习的发展中，大数据和 GPU 做出了很大的贡献。

1. 全连接层

全连接层：相邻层的所有神经元之间都有连接，这称为全连接 (fully-connected)。

全连接层存在什么问题呢？**那就是数据的形状被“忽视”了**。比如，输入数据是图像时，图像通常是高、长、通道方向上的 3 维形状。但是，**向全连接层输入时，需要将 3 维数据拉平为 1 维数据**。

2. CNN 整体结构

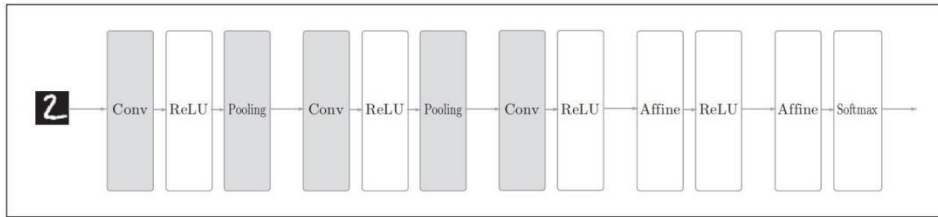


图7-2 基于CNN的网络的例子：新增了 Convolution层和Pooling层(用灰色的方块表示)

CNN 中新增了 Convolution 层和 Pooling 层。CNN 的层的连接顺序是“Convolution - ReLU - (Pooling)” (Pooling 层有时会被省略)。这可以理解为之前的“Affine - ReLU”连接被替换成了“Convolution - ReLU - (Pooling)”连接。

3. 卷积层

全连接层出现的问题：**在全连接层中，相邻层的神经元全部连接在一起，输出的数量可以任意决定。数据的形状被“忽视”了。**比如，输入数据是图像时，图像通常是高、长、通道方向上的 3 维形状。但是，向全连接层输入时，需要将 3 维数据拉平为 1 维数据。实际上，前面提到的使用了 MNIST 数据集的例子中，输入图像就是 1 通道、高 28 像素、长 28 像素的 (1, 28, 28) 形状，但却被排成 1 列，以 784 个数据的形式输入到最开始的 Affine 层。

而卷积层可以保持形状不变。当输入数据是图像时，卷积层会以 3 维数据的形式接收输入数据，并同样以 3 维数据的形式输出至下一层。因此，在 CNN 中，可以 (有可能) 正确理解图像等具有形状的数据。

另外，CNN 中，有时将卷积层的输入输出数据称为特征图 (feature map)。其中，卷积层的输入数据称为输入特征图 (input feature map)，输出数据称为输出特征图 (output feature map)。本书中将“输入输出数据”和“特征图”作为含义相同的词使用。

卷积运算：将各个位置上滤波器的元素和输入的对应元素相乘，然后再求和 (有时将这个计算称为乘积累加运算)。然后，将这个结果保存到输出的对应位置。将这个过程在所有位置都进行一遍，就可以得到卷积运算的输出。

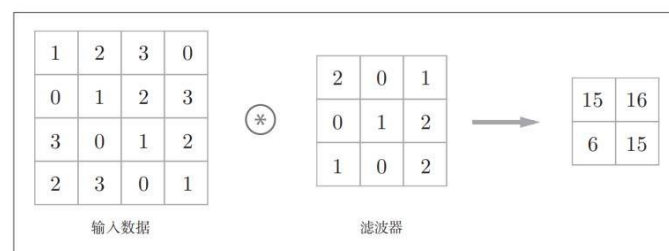


图 7-3 卷积运算的例子：用“⊗”符号表示卷积运算

填充：在进行卷积层的处理之前，有时要向输入数据的周围填入固定的数据 (比如 0 等)，这称为填充 (padding)，是卷积运算中经常会用到的处理。

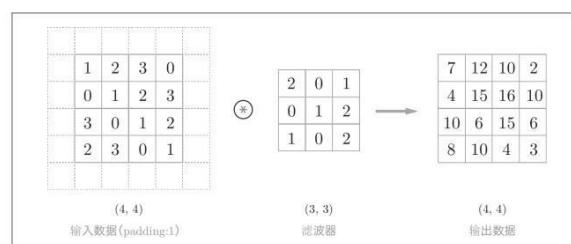


图 7-6 卷积运算的填充处理：向输入数据的周围填入 0 (图中用虚线表示填充，并省略了填充的内容“0”)

步幅：应用滤波器的位置间隔称为步幅 (stride)。之前的例子中步幅都是 1，如果将步幅设为 2，则如图 7-7 所示，应用滤波器的窗口的间隔变为 2 个元素。

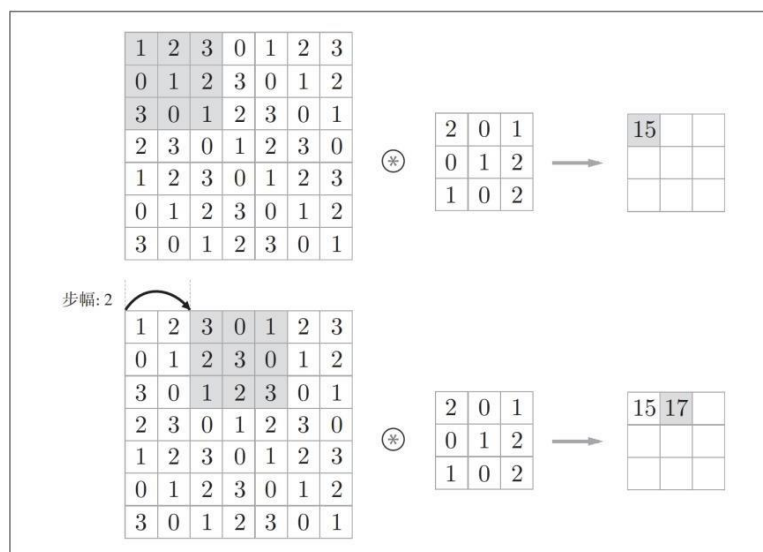


图7-7 步幅为2的卷积运算的例子

这里，假设输入大小为(H, W)，滤波器大小为(FH, FW)，输出大小为(OH, OW)，填充为P，步幅为S。

此时，输出大小可通过式(7.1)进行计算。

这个公式很重要

$$OH = \frac{H + 2P - FH}{S} + 1$$

$$OW = \frac{W + 2P - FW}{S} + 1$$

3 维数据的卷积操作：叠着的位置对应加起来

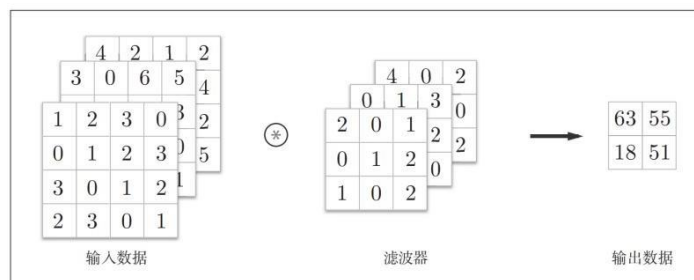


图7-8 对3维数据进行卷积运算的例子

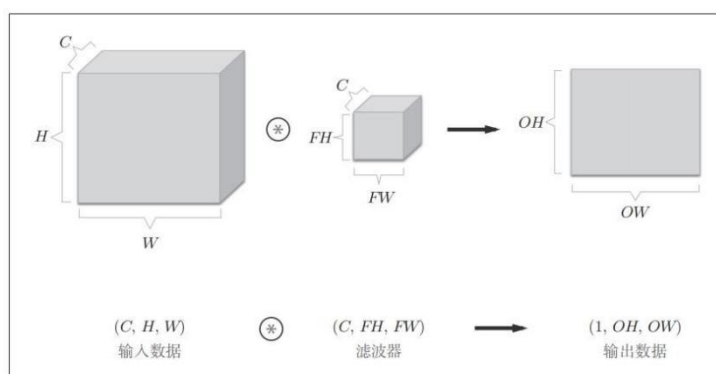


图7-10 结合方块思考卷积运算。请注意方块的形状

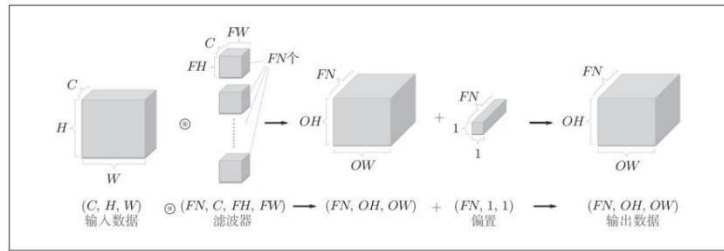


图 7-12 卷积运算的处理流(追加了偏置项)

卷积运算的批处理

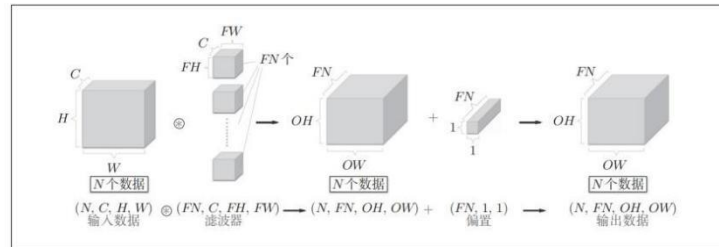


图 7-13 卷积运算的处理流(批处理)

4. 池化处理

特征：

1. 没有要学习的参数：池化层和卷积层不同，没有要学习的参数。池化只是从目标区域中取最大值（或者平均值），所以不存在要学习的参数。
2. 通道数不发生变化：经过池化运算，输入数据和输出数据的通道数不会发生变化。如图 7-15 所示，计算是按通道独立进行的。
3. 对微小的位置变化具有鲁棒性（健壮）：输入数据发生微小偏差时，池化仍会返回相同的结果。因此，池化对输入数据的微小偏差具有鲁棒性。

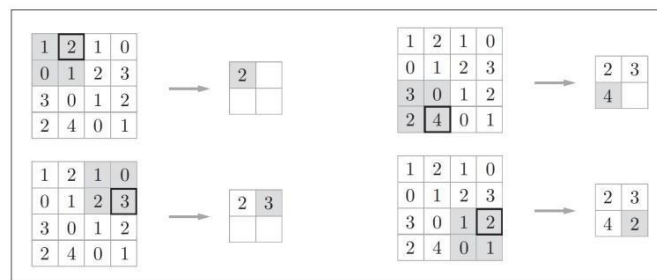


图 7-14 Max 池化的处理顺序

`im2col` 是一个函数，将输入数据展开以适合滤波器（权重）。如图 7-17 所示，对 3 维的输入数据应用 `im2col` 后，数据转换为 2 维矩阵（正确地讲，是把包含批数量的 4 维数据转换成了 2 维数据）。

为了便于观察，将步幅设置得很大，以使滤波器的应用区域不重叠。而在实际的卷积运算中，滤波器的应用区域几乎都是重叠的。在滤波器的应用区域重叠的情况下，使用 `im2col` 展开后，展开后的元素个数会多于原方块的元素个数。因此，使用 `im2col` 的实现存在比普通的实现消耗更多内存的缺点。但是，汇总成一个大矩阵进行计算，对计算机的计算颇有益处。比如，在矩阵计算的库（线性代数库）等中，矩阵计算的实现已被高度最优化，可以高速地进行大矩阵的乘法运算。因此，通过归结到矩阵计算上，可以有效

地利用线性代数库。

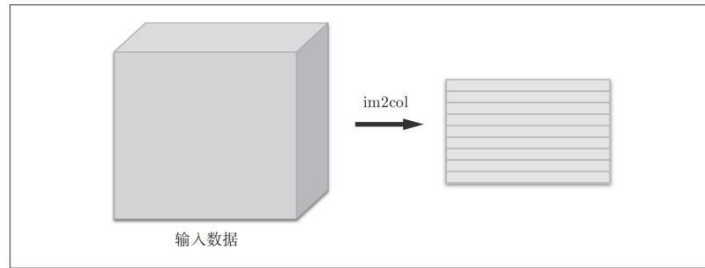


图 7-17 im2col的示意图

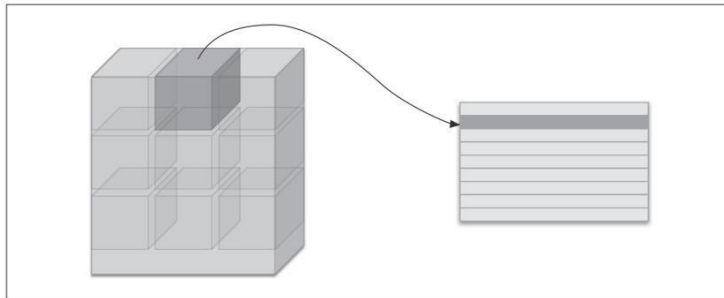


图 7-18 将滤波器的应用区域从头开始依次横向展开为 1 列

为什么要使用 im2col ？

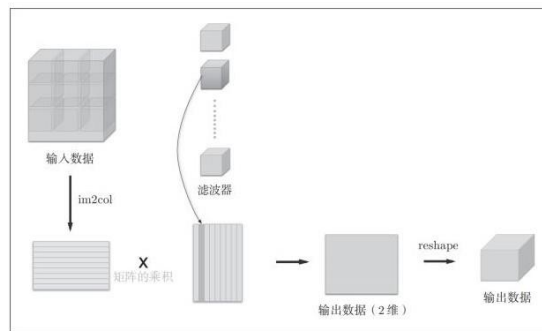


图 7-19 卷积运算的滤波器处理的细节：将滤波器纵向展开为 1 列，并计算和 im2col 展开的数据的矩阵乘积，最后转换 (reshape) 为输出数据的大小

1. 展开输入数据。
2. 求各行的最大值。
3. 转换为合适的输出大小

CNN 可视化：卷积层的滤波器会提取边缘或斑块等原始信息。而刚才实现的 CNN 会将这些原始信息传递给后面的层。

基于分层结构的信息提取：第 1 层的卷积层中提取了边缘或斑块等“低级”信息，那么在堆叠了多层的 CNN 中，各层中又会提取什么样的信息呢？根据深度学习的可视化相关的研究，随着层次加深，提取的信息（正确地讲，是反映强烈的神经元）也越来越抽象。

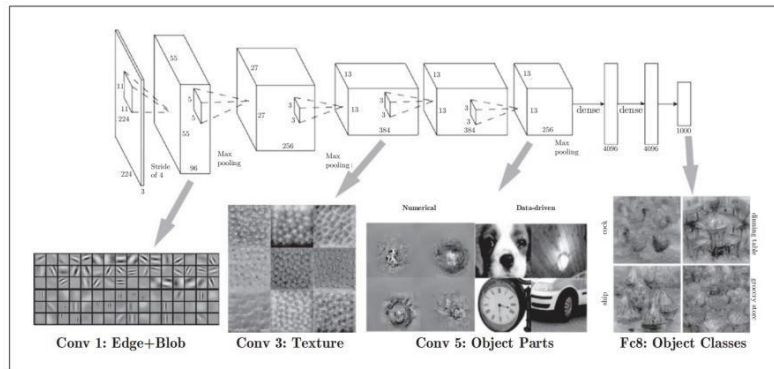


图 7-26 CNN 的卷积层中提取的信息。第 1 层的神经元对边缘或斑块有响应，第 3 层对纹理有响应，第 5 层对物体部件有响应，最后的全连接层对物体的类别(狗或车)有响应(图像引用自文献 [19])

第八章 深度学习

重点内容：

- 什么是数据扩充，怎么进行数据扩充
- 为什么要加深层数
- 注意一下这一章的几个模型

1. 数据扩充

Data Augmentation (数据扩充)：等都助于提高识别精度。尤其是Data Augmentation，虽然方法很简单，但在提高识别精度上效果显著

2. 为什么加深层数

加深层的动机：可以看到层越深，识别性能也越高。其中一个好处就是可以减少网络的参数数量。说得详细一点，就是与没有加深层的网络相比，加深了层的网络可用更少的参数达到同等水平（或者更强）的表现力。这一点结合卷积运算中的滤波器大小来思考就好理解了。

叠加小型滤波器来加深网络的好处：可以减少参数的数量，扩大感受野（receptive field，给神经元施加变化的某个局部空间区域）。并且，通过叠加深层，将 ReLU 等激活函数夹在卷积层的中间，进一步提高了网络的表现力。这是因为向网络添加了基于激活函数的“非线性”表现力，通过非线性函数的叠加，可以表现更加复杂的东西。

加深层的另一个好处就是使学习更加高效。与没有加深层的网络相比，通过加深层，可以减少学习数据，从而高效地进行学习。

迁移学习：将学习完的权重（的一部分）复制到其他神经网络，进行再学习（fine tuning）。比如，准备一个和 VGG 相同结构的网络，把学习完的权重作为初始值，以新数据集为对象，进行再学习。迁移学习在手头数据集较少时非常有效

深度学习进阶：自然语言处理

第二章 自然语言和单词的分布表示

重点内容：

- 什么是共现矩阵
- 降维

1. 共现矩阵、余弦相似度、点互信息、降维

(1) 共现矩阵：各行对应相应的单词向量

	you	say	goodbye	and	i	hello	.
you	0	1	0	0	0	0	0
say	1	0	1	0	1	1	0
goodbye	0	1	0	1	0	0	0
and	0	0	1	0	1	0	0
i	0	1	0	1	0	0	0
hello	0	1	0	0	0	0	1
.	0	0	0	0	0	1	0

图 2-7 用表格汇总各个单词的上下文中包含的单词的频数

(2) 余弦相似度

$\mathbf{x} = (x_1, x_2, x_3, \dots, x_n)$ 和 $\mathbf{y} = (y_1, y_2, y_3, \dots, y_n)$ 两个向量，它们之间的余弦相似度的定义如下式所示。

$$\text{similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \frac{x_1 y_1 + \dots + x_n y_n}{\sqrt{x_1^2 + \dots + x_n^2} \sqrt{y_1^2 + \dots + y_n^2}} \quad (2.1)$$

相似单词排序：

- ① 取出查询词的单词向量。
- ② 分别求得查询词的单词向量和其他所有单词向量的余弦相似度。
- ③ 基于余弦相似度的结果，按降序显示它们的值。

(3) 衡量单词间相似性：点互信息

为了解决这一问题，可以使用点互信息（Pointwise Mutual Information, PMI）这一指标。对于随机变量 x 和 y ，它们的 PMI 定义如下（关于概率，将在 3.5.1 节详细说明）：

$$\text{PMI}(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)} \quad (2.2)$$

其中， $P(x)$ 表示 x 发生的概率， $P(y)$ 表示 y 发生的概率， $P(x, y)$ 表示 x 和 y 同时发生的概率。PMI 的值越高，表明相关性越强。

正点互信息

那就是当两个单词的共现次数为 0 时, $\log_2 0 = -\infty$ 。为了解决这个问题, 实践上我们会使用下述正的点互信息 (Positive PMI, PPMI)。

$$\text{PPMI}(x, y) = \max(0, \text{PMI}(x, y)) \quad (2.6)$$

根据式 (2.6), 当 PMI 是负数时, 将其视为 0, 这样就可以将单词间的相关性表示为大于等于 0 的实数。下面, 我们来实现将共现矩阵转化为

(4) 降维: 在尽量保留“重要信息”的基础上减少向量维度

降维的方法有很多, 这里我们使用奇异值分解 (Singular Value Decomposition, SVD)。SVD 将任意矩阵分解为 3 个矩阵的乘积, 如下式所示:

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T \quad (2.7)$$

如式 (2.7) 所示, SVD 将任意的矩阵 $\mathbf{X}$ 分解为 $\mathbf{U}$ 、 $\mathbf{S}$ 、 $\mathbf{V}$ 这 3 个矩阵的乘积, 其中 $\mathbf{U}$ 和 $\mathbf{V}$ 是列向量彼此正交的正交矩阵, $\mathbf{S}$ 是除了对角线元素以外其余元素均为 0 的对角矩阵。图 2-9 中直观地表示出了这些矩阵。

方法: 奇异值分解

第三章 word2vec

重点内容:

- 什么是 word2vec
- CBOW 模型结构
- 概率表示 CBOW
- skip-gram 模型结构

1 什么是 word2vec

- (5) 要将单词转换为固定长度的向量
- (6) one-hot: 一种方式是将单词转换为 one-hot 表示 (one-hot 向量)。在 one-hot 表示中, 只有一个元素是 1, 其他元素都是 0。

2 CBOW 怎么设计模型

CBOW 模型是根据上下文预测目标词的神经网络 (“目标词”是指中间的单词, 它周围的单词是“上下文”)。通过训练这个 CBOW 模型, 使其能尽可能地进行正确的预测, 我们可以获得单词的分布式表示。

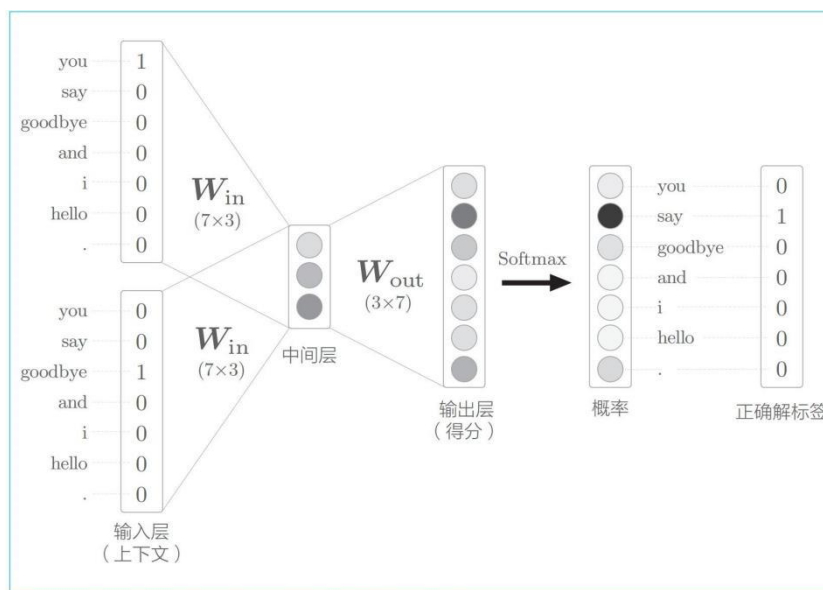


图 3-12 CBOW 模型的示例 (节点值的大小用灰度表示)

3 概率表示的时候 CBOW 是什么样的

下面，我们用数学式来表示当给定上下文 w_{t-1} 和 w_{t+1} 时目标词为 w_t 的概率。使用后验概率，有式 (3.1)：

$$P(w_t | w_{t-1}, w_{t+1}) \quad (3.1)$$

式 (3.1) 表示 “在 w_{t-1} 和 w_{t+1} 发生后， w_t 发生的概率”，也可以解释为 “当给定 w_{t-1} 和 w_{t+1} 时， w_t 发生的概率”。也就是说，CBOW 模型可以建模为式 (3.1)。

CBOW 模型的损失函数只是对式 (3.1) 的概率取 log，并加上负号。顺便提一下，这也称为 负对数似然 (negative log likelihood)。式 (3.2) 是一笔样本数据的损失函数。如果将其扩展到整个语料库，则损失函数可以写为：

$$L = -\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{t-1}, w_{t+1}) \quad (3.3)$$

4 skip-gram 模型，和 CBOW 翻转

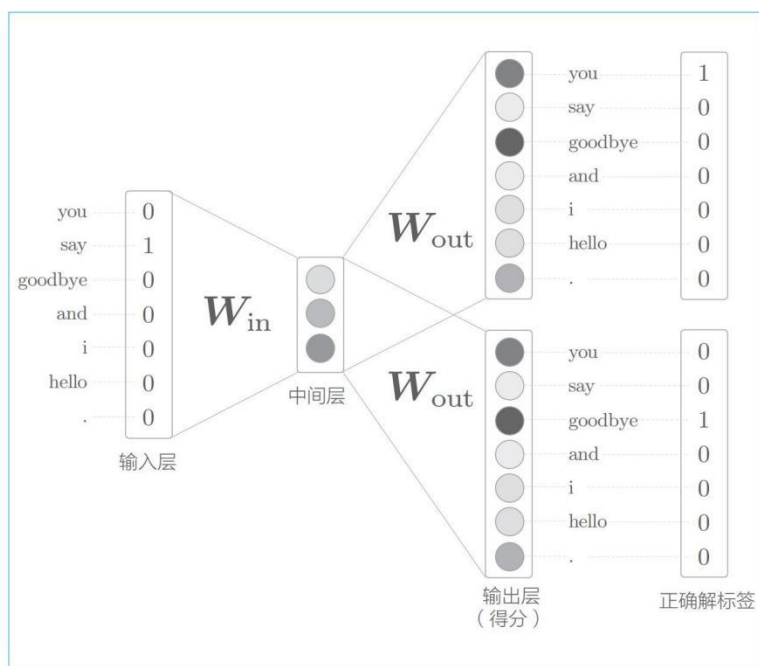


图 3-24 skip-gram 模型的例子

现在，我们使用概率的表示方法来表示 skip-gram 模型。我们来考虑根据中间单词（目标词） w_t 预测上下文 w_{t-1} 和 w_{t+1} 的情况。此时，skip-gram 可以建模为式 (3.4)：

$$P(w_{t-1}, w_{t+1} | w_t) \quad (3.4)$$

式 (3.4) 表示“当给定 w_t 时， w_{t-1} 和 w_{t+1} 同时发生的概率”。这里，在 skip-gram 模型中，假定上下文的单词之间没有相关性（正确地说是假定“条件独立”），将式 (3.4) 如下进行分解：

$$P(w_{t-1}, w_{t+1} | w_t) = P(w_{t-1} | w_t) P(w_{t+1} | w_t) \quad (3.5)$$

通过将式 (3.5) 代入交叉熵误差函数，可以推导出 skip-gram 模型的损失函数：

$$\begin{aligned} L &= -\log P(w_{t-1}, w_{t+1} | w_t) \\ &= -\log P(w_{t-1} | w_t) P(w_{t+1} | w_t) \\ &= -(\log P(w_{t-1} | w_t) + \log P(w_{t+1} | w_t)) \end{aligned} \quad (3.6)$$

skip-gram 模型的损失函数可以表示为式 (3.7)：

$$L = -\frac{1}{T} \sum_{t=1}^T (\log P(w_{t-1} | w_t) + \log P(w_{t+1} | w_t)) \quad (3.7)$$

5 skip-gram 和 CBOW 对比

那么，我们应该使用 CBOW 模型和 skip-gram 模型中的哪一个呢？答案应该是 skip-gram 模型。这是因为，从单词的分布式表示的准确度来看，在大多数情况下，skip-grm 模型的结果更好。特别是随着语料库规模的增大，在低频词和类推问题的性能方面，skip-gram 模型往往会有更好的表现（单词的分布式表

示的评价方法会在 4.4.2 节说明)。此外,就学习速度而言,CBOW 模型比 skip-gram 模型要快。这是因为 skip-gram 模型需要根据上下文数量计算相应个数的损失,计算成本变大。

第四章 word2vec 高速化

重点内容:

- 两个瓶颈是什么
- 如何解决两个瓶颈:创建 Embedding、负采样,要知道这两种如何实现
- 单词向量的评价方法

1. word2vec 计算优化,对输入更新,不需要对全部更新

word2vec 的计算瓶颈:

- 输入层的 one-hot 表示和权重矩阵 W_{in} 的乘积 (4.1 节解决)
- 中间层和权重矩阵 W_{out} 的乘积以及 Softmax 层的计算 (4.2 节解决)

第 1 个问题与输入层的 one-hot 表示有关。这是因为我们用 one-hot 表示来处理单词,随着词汇量的增加,one-hot 表示的向量大小也会增加。比如,在词汇量有 100 万个的情况下,仅 one-hot 表示本身就需要占用 100 万个元素的内存大小。此外,还需要计算 one-hot 表示和权重矩阵 W_{in} 的乘积,这也要花费大量的计算资源。关于这个问题,我们会在 4.1 节中通过引入新的 Embedding 层来解决。

第 2 个问题是中间层之后的计算。首先,中间层和权重矩阵 W_{out} 的乘积需要大量的计算。其次,随着词汇量的增加,Softmax 层的计算量也会增加。关于这些问题,我们将在 4.2 节通过引入 Negative Sampling 这一新的损失函数来解决。下面就让我们通过改进来消除这两个瓶颈。

解决问题一:创建 Embedding

我们创建一个从权重参数中抽取“单词 ID 对应行(向量)”的层,这里我们称之为 Embedding 层。顺便说一句,Embedding 来自“词嵌入”(word embedding)这一术语。也就是说,在这个 Embedding 层存放词嵌入(分布式表示)。

正向传播:Embedding 层的正向传播只是从权重矩阵 W 中提取特定的行,并将该特定行的神经元原样传给下一层

反向传播:在反向传播时,从上一层(输出侧的层)传过来的梯度将原样传给下一层

这里,取出权重梯度 dW ,通过 $dW[idx] = 0$ 将 dW 的元素设为 0 (并不是将 dW 设为 0,而是保持 dW 的形状不变,将它的元素设为 0)。然后,将上一层传来的梯度 $dout$ 写入 idx 指定的行。

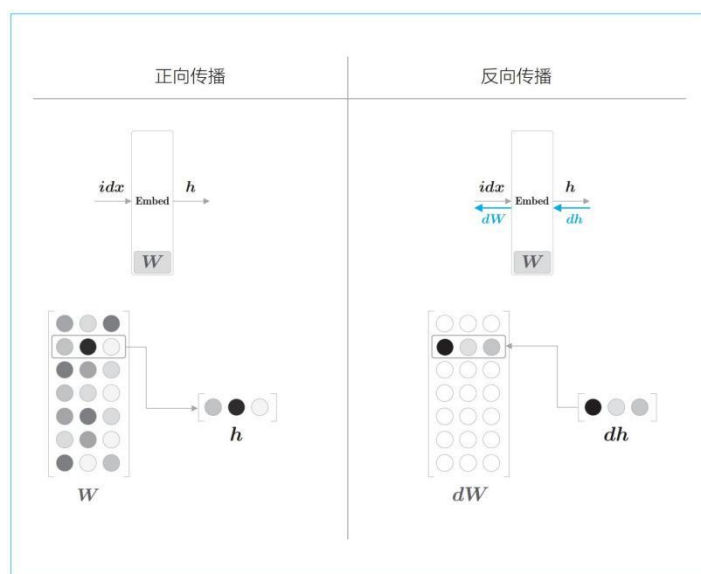


图 4-4 Embedding 层的正向传播和反向传播处理的概要 (Embedding 层记为 Embed)

在刚才的 `backward()` 的实现中，存在一个问题，这一问题发生在 `idx` 的元素出现重复时。比如，当 `idx` 为 `[0, 2, 0, 4]` 时，就会发生图 4-5 中的问题。为了解决这个重复问题，需要进行“加法”，而不是“写入”。

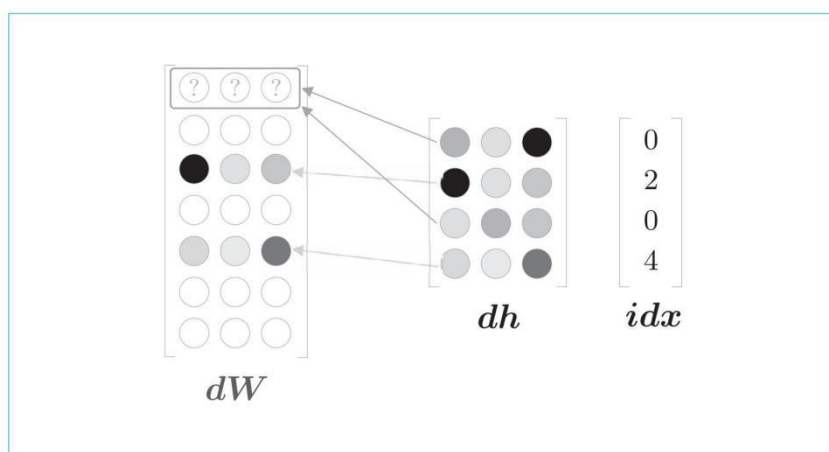


图 4-5 当 `idx` 数组的元素中出现相同的行号时，简单地将 `dh` 的行写入对应位置就会有问题

2. 如果负样本太多，就用采样的方法，看一下负采样的步骤

负采样：下面，我们来解释一下负采样。这个方法的关键思想在于二分类（`binary classification`），更准确地说，是用二分类拟合多分类（`multiclass classification`），这是理解负采样的重点。

输出层的神经元仅有一个。因此，要计算中间层和输出侧的权重矩阵的乘积，只需要提取 `say` 对应的列（单词向量），并用它与中间层的神经元计算内积即可

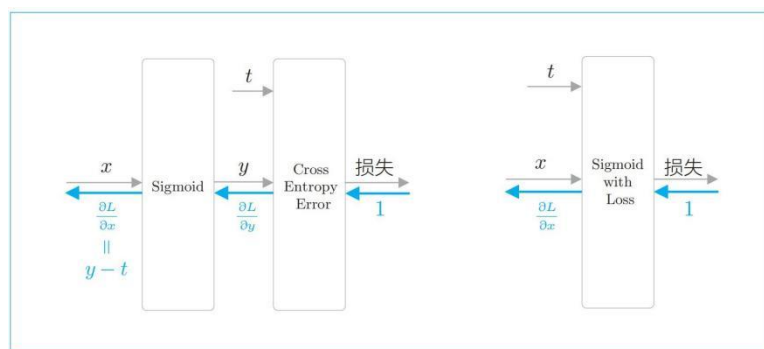


图4-10 Sigmoid层和Cross Entropy Error层的计算图。右图整合为了Sigmoid with Loss层

y 是神经网络输出的概率， t 是正确解标签， $y - t$ 正好是这两个值的差。这意味着，当正确解标签是1时，如果 y 尽可能地接近 1（100%），误差将很小。反过来，如果 y 远离 1，误差将增大。随后，这个误差向前面的层传播，当误差大时，模型学习得多；当误差小时，模型学习得少。

当前的神经网络只是学习了正例 say，但是对 say 之外的负例一无所知。而我们真正要做的事情是，对于正例（say），使 Sigmoid 层的输出接近 1；对于负例（say 以外的单词），使 Sigmoid 层的输出接近0。

如果以所有的负例为对象，词汇量将暴增至无法处理（更何况本章的目的本来就是解决词汇量增加的问题）。为此，作为一种近似方法，我们将选择若干个（5个或者10个）负例。

负采样的采样方法：

基于语料库的统计数据进行采样的方法比随机抽样要好。具体来说，就是让语料库中经常出现的单词容易被抽到，让语料库中不经常出现的单词难以被抽到。基于语料库中单词使用频率的采样方法会先计算语料库中各个单词的出现次数，并将其表示为“概率分布”，然后使用这个概率分布对单词进行采样。

word2vec 中提出的负采样对刚才的概率分布增加了一个步骤。如式 (4.4) 所示，对原来的概率分布取 0.75 次方

$$P'(w_i) = \frac{P(w_i)^{0.75}}{\sum_j^n P(w_j)^{0.75}} \quad (4.4)$$

那么，为什么我们要进行式 (4.4) 的变换呢？这是为了防止低频单词被忽略。更准确地说，通过取 0.75 次方，低频单词的概率将稍微变高。

3. 单词向量的评价方法

经常使用的评价指标有“相似度”和“类推问题”。

单词相似度的评价通常使用人工创建的单词相似度评价集来评估。比如，cat 和 animal 的相似度是 8，cat 和 car 的相似度是 2.....类似这样，用 0 ~ 10 的分数人工地对单词之间的相似度打分。然后，比较人给出的分数和 word2vec 给出的余弦相似度，考察它们之间的相关性。

类推问题的评价是指，基于诸如“king : queen = man : ?”这样的类推问题，根据正确率测量单词的分布式表示的优劣。

第五章 RNN

重点内容

- 理解 RNN 含义
- 注意基础的计算公式
- RNN 如何进行一个简单的单词预测
- RNNlm 基础架构是什么
- 梯度消失和梯度爆炸原因、解决方法
- 整个 LSTM 都非常重要：加入输入门、输出门、遗忘门的整体结构。每个门对应的公式表达要清楚
- 困惑度的概念
- 权重共享的思想

语言模型 (language model) 给出了单词序列发生的概率。具体来说，就是使用概率来评估一个单词序列发生的可能性。

上面介绍的 CBOW 和 skip-gram 只考虑了输入的单词，并没有考虑到单词输入的顺序，我们希望 RNN 考虑到输入的顺序问题。

现在，我们使用数学式来表示语言模型。这里考虑由 m 个单词 $w_1, \dots, w_m$ 构成的句子，将单词按 $w_1, \dots, w_m$ 的顺序出现的概率记为 $P(w_1, \dots, w_m)$ 。因为这个概率是多个事件一起发生的概率，所以称为联合概率。

使用后验概率可以将这个联合概率 $P(w_1, \dots, w_m)$ 分解成如下形式：

$$\begin{aligned}
 P(w_1, \dots, w_m) &= P(w_m | w_1, \dots, w_{m-1}) P(w_{m-1} | w_1, \dots, w_{m-2}) \\
 &\quad \dots P(w_3 | w_1, w_2) P(w_2 | w_1) P(w_1) \\
 &= \prod_{t=1}^m P(w_t | w_1, \dots, w_{t-1})^{\textcircled{1}}
 \end{aligned} \tag{5.4}$$

与表示总和的 $\sum$ (sigma) 相对，式 (5.4) 中的 $\prod$ (pi) 表示所有元素相乘的乘积。如式 (5.4) 所示，联合概率可以由后验概率的乘积表示。

1. RNN 的结构，数据流通环路

RNN 的特征就在于拥有这样一个环路 (或回路)。这个环路可以使数据不断循环。通过数据的循环，RNN 一边记住过去的的数据，一边更新到最新的数据。

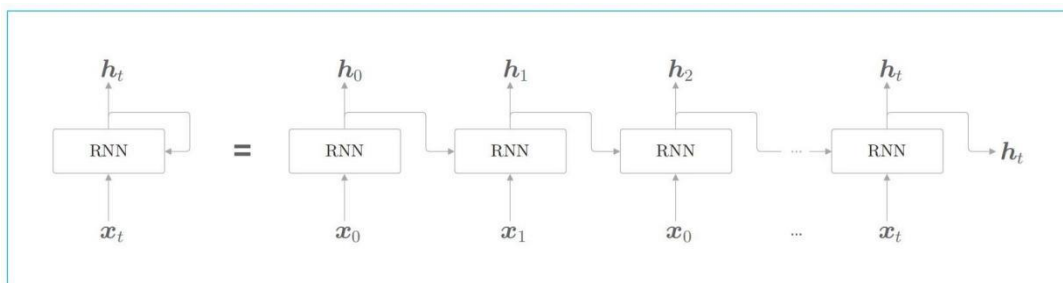


图 5-8 RNN 层的循环的展开

由图 5-8 可以看出，各个时刻的 RNN 层接收传给该层的输入和前一个 RNN 层的输出，然后据此计算当前时刻的输出，此时进行的计算可以用下式表示：

$$h_t = \tanh(h_{t-1}W_h + x_tW_x + b) \quad (5.9)$$

首先说明一下式 (5.9) 中的符号。RNN 有两个权重，分别是将输入 x 转化为输出 h 的权重 W_x 和将前一个 RNN 层的输出转化为当前时刻的输出权重 W_h 。此外，还有偏置 b 。这里， h_{t-1} 和 x_t 都是行向量。

在式 (5.9) 中，首先执行矩阵的乘积计算，然后使用 $\tanh$ 函数（双曲正切函数）变换它们的和，其结果就是时刻 t 的输出 h_t 。这个 h_t 一方面向上输出到另一个层，另一方面向右输出到下一个 RNN 层（自身）。

观察式 (5.9) 可以看出，现在的输出 h_t 是由前一个输出 h_{t-1} 计算出来的。从另一个角度看，这可以解释为，RNN 具有“状态” h ，并以式 (5.9) 的形式被更新。这就是说 RNN 层是“具有状态的层”或“具有存储（记忆）的层”的原因。

BPTT：将循环展开后的 RNN 可以使用（常规的）误差反向传播法。换句话说，可以通过先进行正向传播，再进行反向传播的方式求目标梯度。因为这里的误差反向传播法是“按时间顺序展开的神经网络的误差反向传播法”，所以称为 **Backpropagation Through Time（基于时间的反向传播）**，简称 BPTT。

Truncated BPTT：在处理长时序数据时，通常的做法是将网络连接截成适当的长度。具体来说，就是将时间轴方向上过长的网络在合适的位置进行截断，从而创建多个小型网络，然后对截出来的小型网络执行误差反向传播法，这个方法称为 **Truncated BPTT（截断的 BPTT）**

在 Truncated BPTT 中，网络连接被截断，但严格地讲，只是网络的反向传播的连接被截断，正向传播的连接依然被维持，这一点很重要。也就是说，正向传播的信息没有中断地传播。与此相对，反向传播则被截断为适当的长度，以被截出的网络为单位进行学习。

虽然 Truncated BPTT 的原理非常简单，但是关于数据的输入方法有几个需要注意的地方。具体而言，一是要按顺序输入数据，二是要平移各批次（各样本）输入数据的开始位置。

2. RNNLM 的结构

我们将基于 RNN 的语言模型称为 RNNLM（RNN Language Model，RNN 语言模型）。

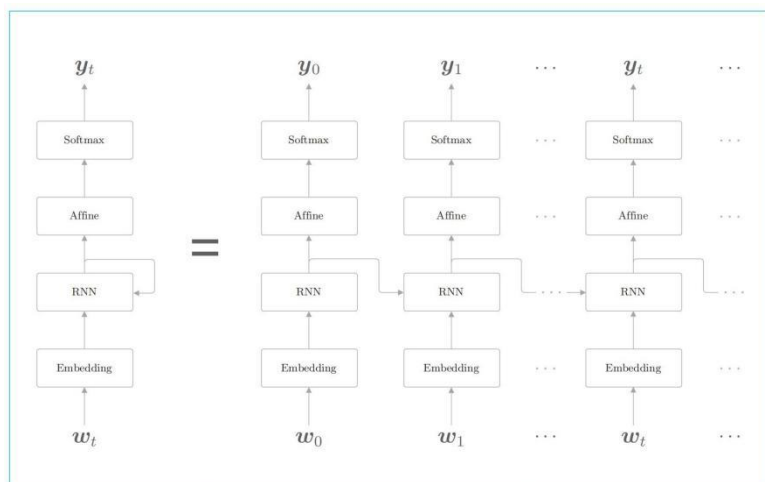


图 5-25 RNNLM 的网络图（左图是展开前，右图是展开后）

3. 困惑度

困惑度 (perplexity) 常被用作评价语言模型的预测性能的指标。

困惑度表示“概率的倒数” (这个解释在数据量为 1 时严格一致)。为了说明概率的倒数，我们仍旧考虑“you say goodbye and i say hello.” 这一语料库。假设在向语言模型“模型 1”传入单词 you 时会输出图5-32 的左图所示的概率分布。此时，下一个出现的单词是 say 的概率为 0.8，这是一个相当不错的预测。取这个概率的倒数，可以计算出困惑度为 $1/0.8 = 1.25$ 。

4. 梯度消失、梯度爆炸

RNN 之所以不擅长学习时序数据的长期依赖关系，是因为 BPTT 会发生梯度消失和梯度爆炸的问题。

梯度爆炸：可知梯度的大小随时间步长呈指数级增加，这就是梯度爆炸 (exploding gradients)。如果发生梯度爆炸，最终就会导致溢出，出现 NaN (Not a Number，非数值) 之类的值。

梯度消失：梯度呈指数级减小，这就是梯度消失 (vanishing gradients)。如果发生梯度消失，梯度将迅速变小。一旦梯度变小，权重梯度不能被更新，模型就会无法学习长期的依赖关系。

5. 为什么会发生梯度消失、梯度爆炸

因为矩阵 W_h 被反复乘了 T 次。如果 W_h 是标量，则问题将很简单：当 W_h 大于 1 时，梯度呈指数级增加；当 W_h 小于 1 时，梯度呈指数级减小。那么，如果 W_h 不是标量，而是矩阵呢？此时，矩阵的奇异值将成为指标。简单而言，矩阵的奇异值表示数据的离散程度。根据这个奇异值 (更准确地说是多个奇异值中的最大值) 是否大于 1，可以预测梯度大小的变化。如果奇异值的最大值大于 1，则可以预测梯度很有可能呈指数级增加；而如果奇异值的最大值小于 1，则可以判断梯度会呈指数级减小。

6. 如何解决梯度爆炸：梯度裁剪

解决梯度爆炸有既定的方法，称为**梯度裁剪 (gradients clipping)**。这是一个非常简单的方法，它的伪代码如下所示：

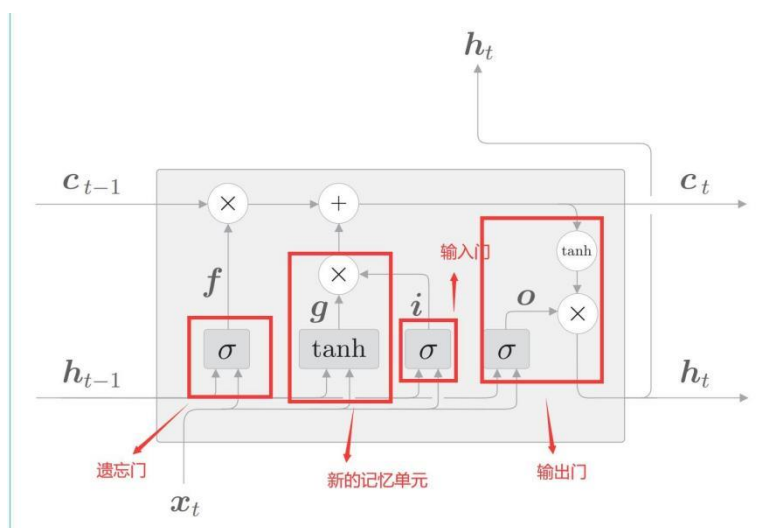
$$\text{if } \|\hat{g}\| \geq \text{threshold}: \\ \hat{g} = \frac{\text{threshold}}{\|\hat{g}\|} \hat{g}$$

这里假设可以将神经网络用到的所有参数的梯度整合成一个，并用符号 $\hat{g}$ 表示。另外，将阈值设置为 threshold 。此时，如果梯度的 L2 范数 $\|\hat{g}\|$ 大于或等于阈值，就按上述方法修正梯度，这就是梯度裁剪。如你所见，虽然这个方法很简单，但是在许多情况下效果都不错。

7. 如何解决梯度消失：LSTM 整个 LSTM 都非常重要：加入输入门、输出门、遗忘门的整体结构。每个门对应的公式表达要清楚

LSTM 还有路径 c 。这个 c 称为记忆单元 (或者简称为“单元”)，相当于 LSTM 专用的记忆部门。记忆单元的特点是，仅在 LSTM 层内部接收和传递数据。也就是说，记忆单元在 LSTM 层内部结束工作，不向其他层输出。而 LSTM 的隐藏状态 h 和 RNN 层相同，会被 (向上) 输出到其他层。

结构牢记！！！！



输出门：

记忆单元 c_t 仅仅应用了 $\tanh$ 函数。这里考虑对 $\tanh(c_t)$ 施加门。换句话说，针对 $\tanh(c_t)$ 的各个元素，调整它们作为下一时刻的隐藏状态的重要程度。由于这个门管理下一个隐藏状态 h_t 的输出，所以称为输出门（output gate）。

输出门的开合程度（流出比例）根据输入 x_t 和上一个状态 h_{t-1} 求出。此时进行的计算如式（6.1）所示。这里在使用的权重参数和偏置的上标上添加了 output 的首字母 o。之后，我们也将使用上标表示门。另外，sigmoid 函数用 $\sigma()$ 表示。

$$o = \sigma(x_t W_x^{(o)} + h_{t-1} W_h^{(o)} + b^{(o)}) \quad (6.1)$$

遗忘门：我们在记忆单元 c_{t-1} 上添加一个忘记不必要记忆的门，这里称为遗忘门

在图 6-16 中，将遗忘门进行的一系列计算表示为 σ ，其中有遗忘门专用的权重参数，此时的计算如下：

$$f = \sigma(x_t W_x^{(f)} + h_{t-1} W_h^{(f)} + b^{(f)}) \quad (6.3)$$

遗忘门的输出 f 可以由式（6.3）求得。然后， c_t 由这个 f 和上一个记忆单元 c_{t-1} 的对应元素的乘积求得（ $c_t = f \odot c_{t-1}$ ）。

记忆单元：现在我们还想向这个记忆单元添加一些应当记住的新信息，为此我们添加新的 $\tanh$ 节点

如图 6-17 所示，基于 $\tanh$ 节点计算出的结果被加到上一时刻的记忆单元 c_{t-1} 上。这样一来，新的信息就被添加到了记忆单元中。这个 $\tanh$ 节点的作用不是门，而是将新的信息添加到记忆单元中。因此，它不用 sigmoid 函数作为激活函数，而是使用 $\tanh$ 函数。 $\tanh$ 节点进行的计算如下所示：

$$g = \tanh(x_t W_x^{(g)} + h_{t-1} W_h^{(g)} + b^{(g)}) \quad (6.4)$$

输入门：

输入门判断新增信息 g 的各个元素的价值有多大。输入门不会不经考虑就添加新信息，而是会对要添加的信息进行取舍。换句话说，输入门会添加加权后的新信息。

在图 6-18 中，用 σ 表示输入门，用 i 表示输出，此时进行的计算如下所示：

$$i = \sigma(x_t W_x^{(i)} + h_{t-1} W_h^{(i)} + b^{(i)}) \quad (6.5)$$

为什么 LSTM 不会引起梯度消失？

而“ $\times$ ”节点的计算并不是矩阵乘积，而是对应元素的乘积（阿达玛积）。顺便说一下，在之前的 RNN 的反向传播中，我们使用相同的权重矩阵重复了多次矩阵乘积计算，由此导致了梯度消失（或梯度爆炸）。而这里的 LSTM 的反向传播进行的不是矩阵乘积计算，而是对应元素的乘积计算，而且每次都会基于不同的门值进行对应元素的乘积计算。这就是它不会发生梯度消失（或梯度爆炸）的原因。

所有的计算过程：

$$\begin{aligned} f &= \sigma(x_t W_x^{(f)} + h_{t-1} W_h^{(f)} + b^{(f)}) \\ g &= \tanh(x_t W_x^{(g)} + h_{t-1} W_h^{(g)} + b^{(g)}) \\ i &= \sigma(x_t W_x^{(i)} + h_{t-1} W_h^{(i)} + b^{(i)}) \\ o &= \sigma(x_t W_x^{(o)} + h_{t-1} W_h^{(o)} + b^{(o)}) \end{aligned} \quad (6.6)$$

$$c_t = f \odot c_{t-1} + g \odot i \quad (6.7)$$

$$h_t = o \odot \tanh(c_t) \quad (6.8)$$

Dropout 在 LSTM 上的使用：

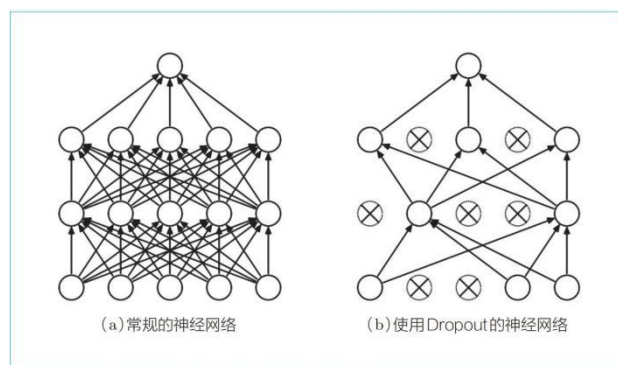


图 6-30 Dropout 的概念图(参考文献[9]): 左边是常规的神经网络, 右边是使用了 Dropout 的网络

地积累。考虑到噪声的积累, 最好不要在时间轴方向上插入 Dropout。因此, 如图 6-33 所示, 我们在深度方向(垂直方向)上插入 Dropout 层。

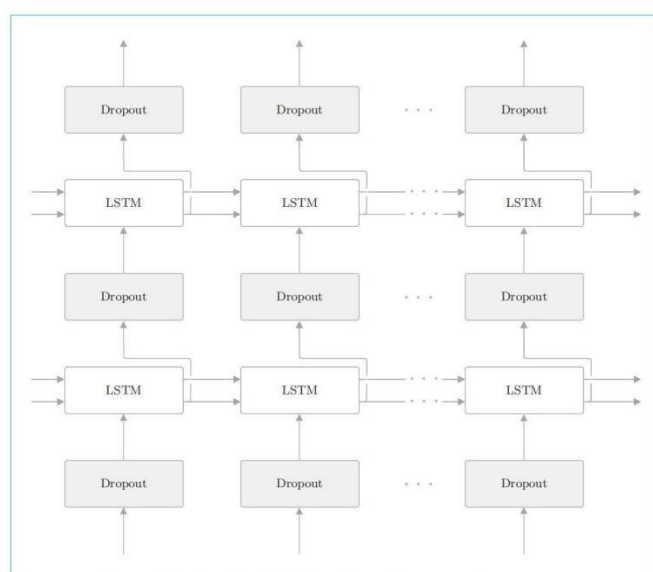


图 6-33 好的例子: 在深度方向(垂直方向)上插入 Dropout 层

这样一来, 无论沿时间方向(水平方向)前进多少, 信息都不会丢失。Dropout 与时间轴独立, 仅在深度方向(垂直方向)上起作用。

8. 权重共享

绑定(共享) Embedding 层和 Affine 层的权重的技巧在于权重共享。通过在这两个层之间共享权重, 可以大大减少学习的参数数量。

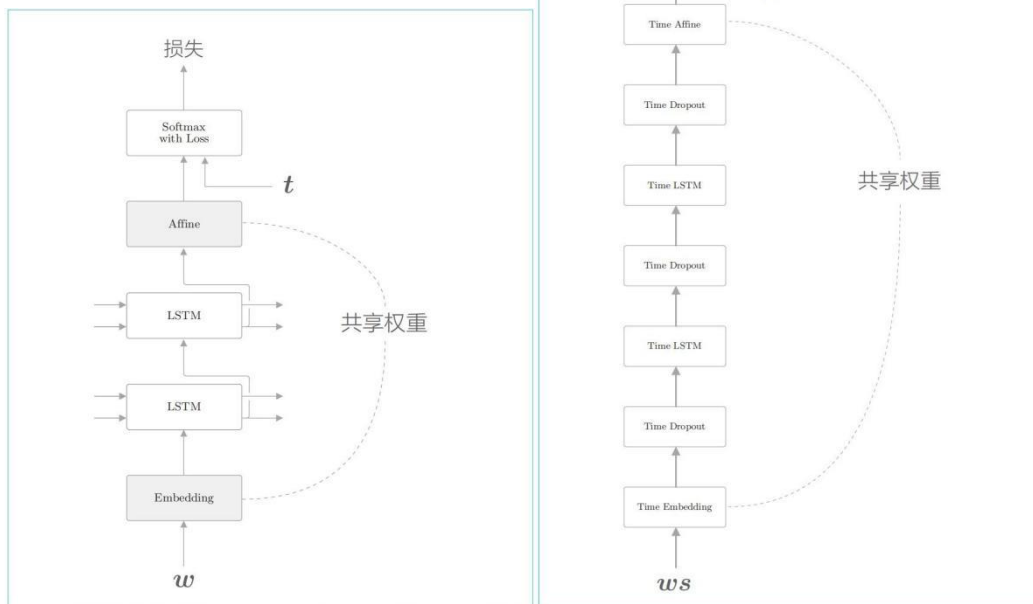


图 6-35 语言模型中共享权重的例子: Embedding 层和 Softmax 前的 Affine 层共享权重 图 6-36 BetterRnnlm 类的网络结构

第七章 基于 RNN 生成文本

重点内容

- seq2seq 理解基础含义
- 知道 Encoder 和 Decoder
- Reverse
- Peeky

seq2seq：将一个时序数据转换为另一个时序数据。

1. 输入一个词，给出概率最高的下一个词，再输入再输出

它如何生成下一个新单词呢？

一种可能的方法是选择概率最高的单词。在这种情况下，因为选择的是概率最高的单词，所以结果能唯一确定。也就是说，这是一种“确定性的”方法。另一种方法是“概率性地”进行选择。根据概率分布进行选择，这样概率高的单词容易被选到，概率低的单词难以被选到。在这种情况下，被选到的单词（被采样到的单词）每次都不一样。

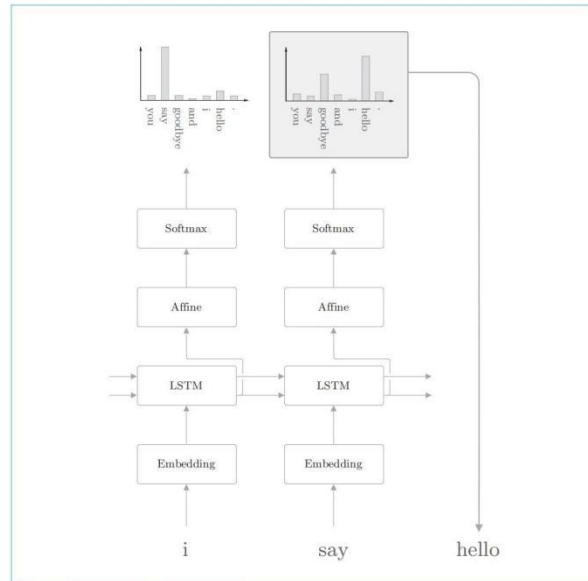


图 7-4 重复概率分布的输出和采样

- 知道 Encoder 和 Decoder，编码器结构，输出 h 是一个固定长度向量，解码器有一个开始符结束符
seq2seq 模型也称为 Encoder-Decoder 模型。顾名思义，这个模型有两个模块——Encoder（编码器）和 Decoder（解码器）。编码器对输入数据进行编码，解码器对被编码的数据进行解码。

编码就是将任意长度的文本转换为一个固定长度的向量

解码器“处理”这个编码好的向量，从而生成目标文本

<eos> 这一分隔符（特殊符号）。这个分隔符被用作通知解码器开始生成文本的信号

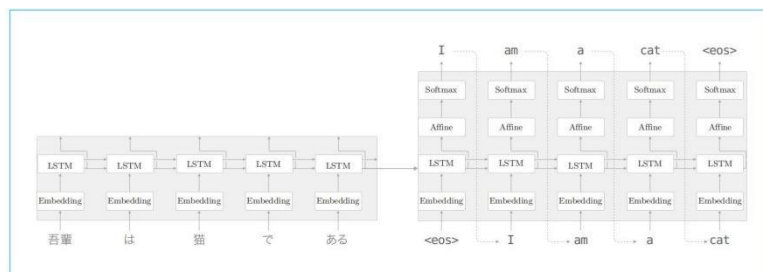


图 7-9 seq2seq 的整体的层结构

如图 7-9 所示，seq2seq 由两个 LSTM 层构成，即编码器的 LSTM 和解码器的 LSTM。此时，LSTM 层的隐藏状态是编码器和解码器的“桥梁”。在正向传播时，编码器的编码信息通过 LSTM 层的隐藏状态传递给解码器；在反向传播时，解码器的梯度通过这个“桥梁”传递给编码器。

padding：在基于 mini-batch 学习可变长度的时序数据时，最简单的方法是使用填充（padding）。所谓填充，就是用无效（无意义）数据填入原始数据，从而使数据长度对齐

3. 翻转输入数据 Reverse

为什么反转数据后，学习进展变快，精度提高了呢？虽然理论上不是很清楚，但是直观上可以认为，反转数据后梯度的传播可以更平滑。如此，因为通过反转，输入语句的开始部分和对应的转换后的单词之间的距离变近（这样的情况变多），所以梯度的传播变得更容易，学习效率也更高。不过，在反转输入数据后，单词之间的“平均”距离并不会发生改变。

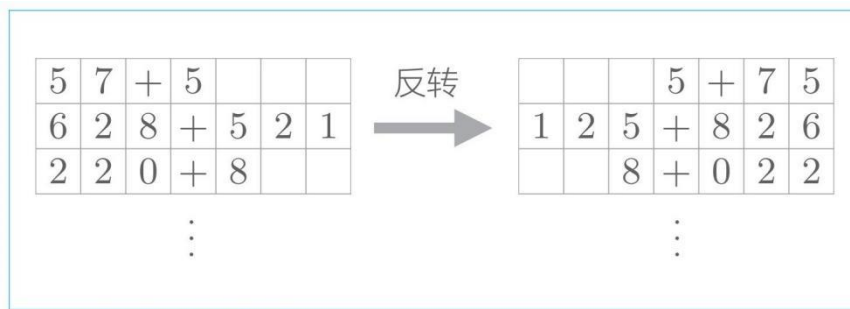


图 7-23 反转输入数据的例子

4. 偷窥 Peeky

编码器将输入语句转换为固定长度的向量 h ，这个 h 集中了解码器所需的全部信息。也就是说，它是解码器唯一的信息源。当前的 seq2seq 只有最开始时刻的 LSTM 层利用了 h 。我们能更加充分地利用这个 h 吗？

为了达成该目标，seq2seq 的第二个改进方案就应运而生了。具体来说，就是将这个集中了重要信息的编码器的输出 h 分配给解码器的其他层。

改进前：

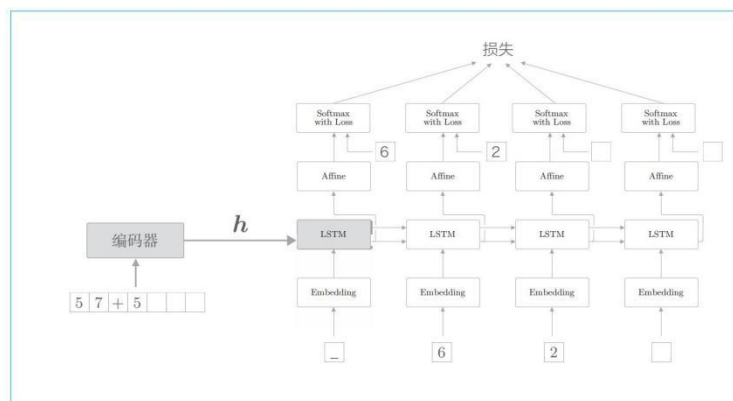


图 7-25 改进前：只有最开始的 LSTM 层接收编码器的输出 h

改进后：

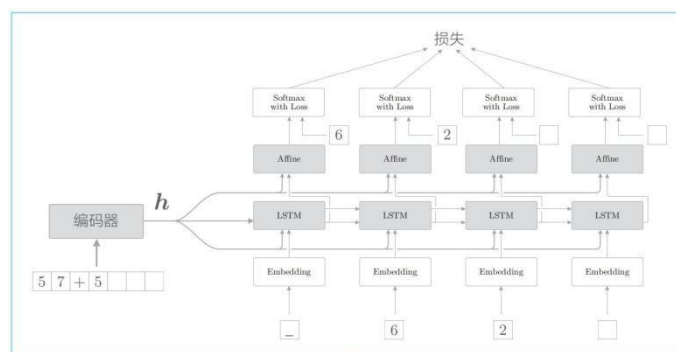


图 7-26 改进后：将编码器的输出 h 分配给所有时刻的 LSTM 层和 Affine 层

如图 7-26 所示，将编码器的输出 h 分配给所有时刻的 Affine 层和 LSTM 层。比较图 7-26 和图 7-25 可知，之前 LSTM 层专用的重要信息 h

在图 7-26 中，有两个向量同时被输入到了 LSTM 层和 Affine 层，这实际上表示两个向量的拼接（concatenate）

重要内容：

- Attention 主要思想
- 编码器的改进：hs 加权求和
- 模型 attention weight 是怎么实现的，attention sum 是什么意思，用 hs 的每一个行点乘 h 得到 s，再由 softmax 函数得到权重

1. 对编码器的改进，hs (加权求和)

这里我们所做的改进只是将编码器的全部时刻的隐藏状态取出来而已。通过这个小改动，编码器可以

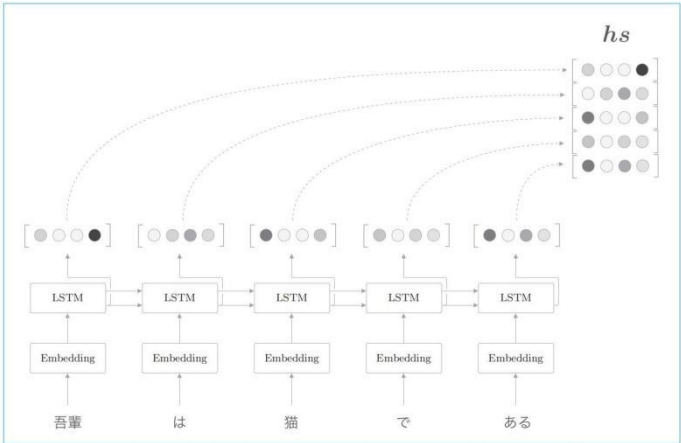


图 8-2 使用编码器各个时刻(各个单词)的 LSTM 层的隐藏状态(这里表示为 hs)

根据输入语句的长度，成比例地编码信息。

上一章的解码器只用了编码器的 LSTM 层的最后的隐藏状态。如果使用 hs ，则只提取最后一行，再将其传递给解码器。下面我们改进解码器，以便能够使用全部 hs 。

可否将“选择”这一操作换成可微分的运算呢？实际上，解决这个问题的思路很简单（但是，就像哥伦布蛋一样，第一个想到是很难的）。这个思路就是，与其“单选”，不如“全选”。如图 8-7 所示，我们另行计算表示各个单词重要度（贡献值）的权重。

如图 8-7 所示，这里使用了表示各个单词重要度的权重（记为 a ）。此时， a 像概率分布一样，各元素是 0.0 ~ 1.0 的标量，总和是 1。然后，计算这个表示各个单词重要度的权重和单词向量 hs 的加权和，可以获得目标向量。这一系列计算如图 8-8 所示。

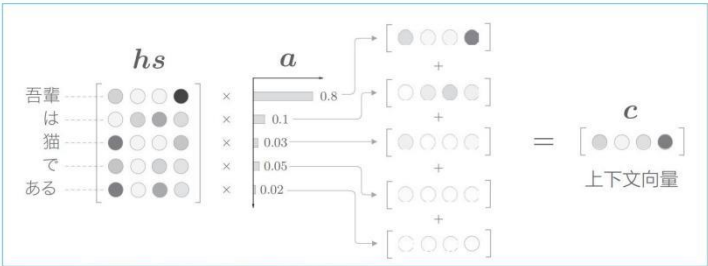


图 8-8 通过计算加权和，可以得到上下文向量

2. 模型 attention weight 是怎么实现的，attention sum 是什么意思，用 hs 的每一个行点乘 h 得到 s ，再由 softmax 函数得到权重

有了表示各个单词重要度的权重 a ，就可以通过加权和获得上下文向量。那么，怎么求这个 a 呢？
当然不需要我们手动指定，我们只需要做好让模型从数据中自动学习它的准备工作。

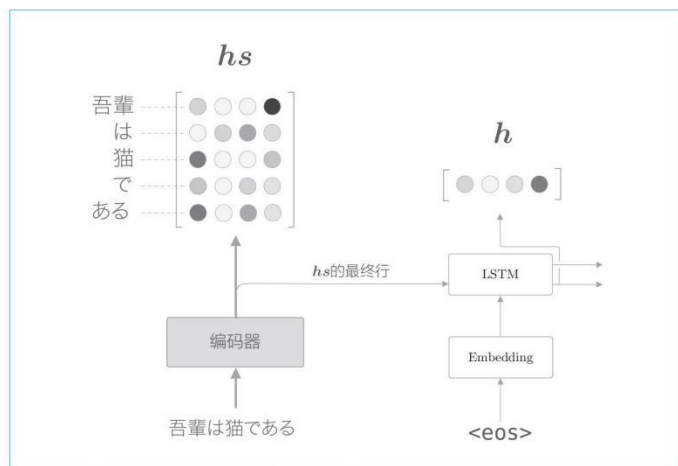


图 8-12 解码器第 1 个 LSTM 层的隐藏状态向量

在图 8-12 中，用 h 表示解码器的 LSTM 层的隐藏状态向量。此时，我们的目标是用数值表示这个 h 在多大程度上和 hs 的各个单词向量“相似”。有几种方法可以做到这一点，这里我们使用最简单的向量内积。顺便说一下，向量 $a = (a_1, a_2, \dots, a_n)$ 和向量 $b = (b_1, b_2, \dots, b_n)$ 的内积为：

$$a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n \quad (8.1)$$

式 (8.1) 的含义是两个向量在多大程度上指向同一方向，因此使用内积作为两个向量的“相似度”是非常自然的选择。

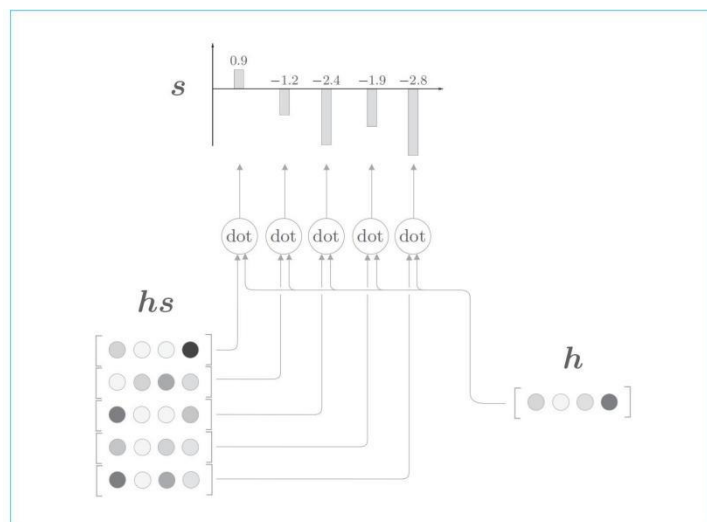


图 8-13 基于内积计算 hs 的各行与 h 的相似度 (内积用 dot 节点表示)

如图 8-13 所示，这里通过向量内积算出 h 和 hs 的各个单词向量之间的相似度，并将其结果表示为 s 。不过，这个 s 是正规化之前的值，也称为得分。接下来，使用老一套的 Softmax 函数对 s 进行正规化 (图 8-14)。

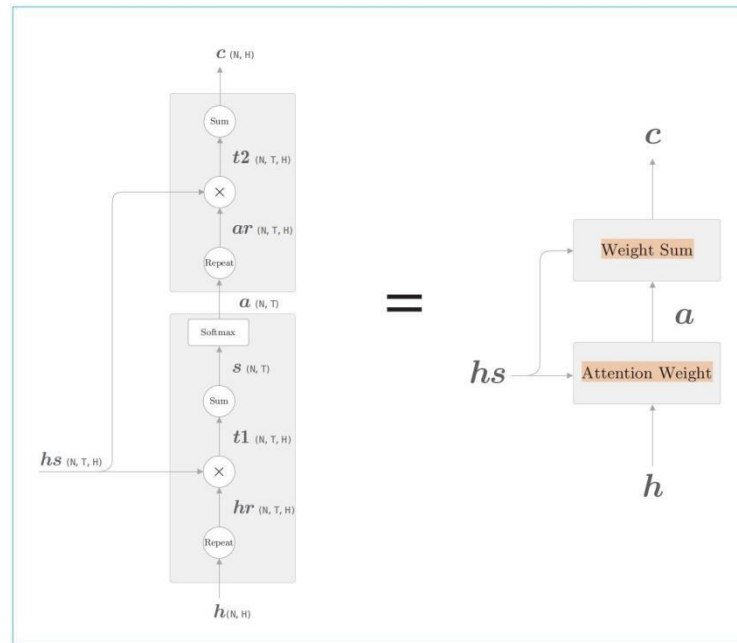


图 8-16 计算上下文向量的计算图

图 8-16 显示了用于获取上下文向量 c 的计算图的全貌。我们已经分为 Weight Sum 层和 Attention Weight 层进行了实现。重申一下，这里进行的计算是：Attention Weight 层关注编码器输出的各个单词向量 hs ，并计算各个单词的权重 a ；然后，Weight Sum 层计算 a 和 hs 的加权和，并输出上下文向量 c 。我们将进行这一系列计算的层称为 Attention 层（图 8-17）。

考试不涉及代码

题型：名词解释、简答、综合

2022 考试题

一、名词解释

1. 全连接层

每个神经元都与前一层的所欲神经元相连形成一个完全连接的图结构

2. 感知器

一种简单的人工神经元模型，由一个或多个输入节点，一个权重向量和一个激活函数组成

3. 梯度爆炸

梯度大小随时间步长指数增加

4. epoch

epoch 是一个单位。一个 epoch 表示学习中所有训练数据均被使用过一次时的更新次数。

二、简答题

1. rnn 梯度消失的原因和解决方法

梯度消失：矩阵 W_h 被反复乘了 T 次。如果矩阵奇异值的最大值大于 1，则可以预测梯度很有可能会呈指数级增加；而如果奇异值的最大值小于 1，则可以判断梯度会呈指数级减小

解决方法：使用 Gated RNN，即 LSTM

2.cbow 和 skipgram 的区别及各自优点

区别：CBOW 根据上下文预测目标词的神经网络，skip-gram 根据当前词汇预测周围的多个词汇

各自优点：

CBOW 学习速度快，计算成本小

skip 性能好，计算的准确度高

3.损失函数的定义，列举两个损失函数

神经网络的学习中所用的指标称为**损失函数**

两个损失函数：均方误差（MSE）、交叉熵损失

可以用作损失函数的函数有很多，其中最有名的是均方误差（mean squared error）。均方误差如下式所示。

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2 \quad (4.1)$$

除了均方误差之外，交叉熵误差（cross entropy error）也经常被用作损失函数。交叉熵误差如下式所示。

$$E = - \sum_k t_k \log y_k \quad (4.2)$$

4.rnn 和传统人工神经网络训练的异同

RNN 具有因此状态 h ，并且不断被更新，就是说 RNN 具有状态层或具有存储状态的层，RNN 可以在记住以往信息的同时学习新的信息

5.relu 和sigmoid 的共性和区别

共性：都是激活函数，都用于决定如何来激活输入信号的总和，都是非线性函数

区别：

1. 方程式不同

2. 一个是连续的一个不连续

3. relu 更简单，可以减少计算量，一定程度解决梯度消失问题

6.比较 gan 和dcgan,简述 gan 存在的问题和 dcgan 的改进？第一本书最后一章

GAN（Generative Adversarial Network）是一种生成模型，由生成器（Generator）和判别器（Discriminator）组成。它们通过对抗训练的方式相互竞争，从而使生成器逐渐学习生成逼真的数据样本。

DCGAN（Deep Convolutional GAN）是对传统 GAN 的改进，通过引入卷积神经网络（CNN）结构来处理图像数据。相比于传统的全连接层网络，DCGAN 利用了卷积层和转置卷积层，更适用于图像生成任务

三、综合题

$f(a,b,c)=3(a+bc)$ 画出计算图和反向传播过程。

$f(a,b,c) = 3(a+bc)$ 计算图和反向传播

